

Università di Firenze
Scuola di Scienze Matematiche Fisiche e Naturali
Laurea Magistrale in Data Science, Calcolo Scientifico & Intelligenza
Artificiale

Apprendimento per Rinforzo Multi-Agente basato su Reti Neurali a Grafo per il Controllo Semaforico

**[DOCUMENTO DI LAVORO –
QUASI-UFFICIALE]**

Capitoli 1, 2 e 5 in versione rivista; capitoli 3 e 4 identici a main.pdf

Relatore

Daniele Castellana

Candidato

Alessio Minati

Anno Accademico 2025/2026

Sommario

Un incrocio semaforico deve garantire un flusso regolare dei veicoli che vi transitano. Le soluzioni classiche affrontano il problema attraverso un piano a tempi fissi, con fasi di durata prestabilita calcolata su stime di traffico medie, oppure attraverso un controllo attuato dai sensori, che concede il verde a una direzione solo quando un sensore rileva la presenza di un veicolo in coda.

Quando gli incroci da controllare sono più di uno, emerge un problema ulteriore: le decisioni prese a un incrocio devono essere coordinate con quelle degli incroci vicini, perché il traffico smaltito da un'intersezione si riversa in parte in quello in entrata ad un'altra intersezione. Sistemi come SCOOT e SCATS rispondono a questa esigenza coordinando i piani semaforici di un'intera area urbana sulla base del traffico osservato in tempo reale.

Entrambe le famiglie di soluzioni si basano però su euristiche definite manualmente e restano per questo poco flessibili di fronte a condizioni di traffico diverse da quelle previste in fase di progettazione. Il reinforcement learning offre un'alternativa, apprendendo una politica di controllo dall'esperienza simulata invece di codificarla esplicitamente. MetaSTGAT (Wang et al., 2022) ne è un esempio, con un agente RL per intersezione coordinato da una *graph attention network* i cui pesi sono generati dinamicamente da un modulo di meta-learning condizionato su caratteristiche locali della rete stradale. Il lavoro originale si concentra però sull'efficienza aggregata della rete, senza verificare se il controllore generalizzi a topologie mai osservate in addestramento né se il vantaggio ottenuto sia distribuito equamente tra le direzioni di traffico servite.

Questo lavoro replica da zero l'architettura e la procedura di training di MetaSTGAT, estendendola esplicitamente su questi due assi, equità e generalizzazione, su un ambiente costruito su CITYFLOW con reti a griglia di tre dimensioni (4×4 , 5×5 , 6×6) e traffico generato parametricamente. Ogni differenza introdotta rispetto alla procedura originale è isolata da un sistema di preset di ablazione e motivata singolarmente.

I risultati sperimentali confrontano le baseline classiche (Fixed-Time, MaxPressu-

re) con tre varianti del modello proposto che pesano diversamente nella ricompensa la componente di equità, con uno studio di ablazione completo e con un confronto tra tre meccanismi di comunicazione spaziale alternativi. Il termine di ricompensa dedicato all'equità risulta essere utile per il raggiungimento della stessa e il meccanismo di comunicazione spaziale scelto si conferma efficace per l'ottenimento di un buon coordinamento tra gli incroci della rete.

Indice

Sommario	i
Acronimi e Notazione	x
1 Introduzione	1
1.1 Motivazioni	1
1.2 Obiettivi e Contributi di questo lavoro	5
1.3 Struttura della tesi	6
2 Background	8
2.1 Struttura del capitolo	8
2.2 Reti Neurali	9
2.3 Reinforcement Learning	11
2.3.1 Il paradigma agente-ambiente	11
2.3.2 Processo decisionale di Markov	12
2.3.3 Apprendimento model-based e model-free	13
2.3.4 Q-learning e Deep Q-Network	13
2.3.5 Esplorazione ε -greedy	15
2.3.6 Estensioni usate in questo lavoro	15
2.3.7 Apprendimento multi-agente	16
2.4 Reti Neurali Ricorrenti	18
2.5 Grafi e Graph Neural Network	20
2.5.1 Cos'è un grafo	20
2.5.2 Message passing	21
2.5.3 Invarianza e equivarianza per permutazione	23
2.5.4 Reti spatio-temporali su grafo	24
2.5.5 Pesi generati da meta-learning	25
2.5.6 Propagazione a lungo raggio	25
2.6 Traffic Signal Control come problema	26
2.6.1 Vocabolario di base	26

2.6.2	Tre generazioni di controllo	27
2.6.3	Simulazione	30
3	Metodologia	32
3.1	Formalizzazione del problema	32
3.2	Ambiente di simulazione	34
3.2.1	Struttura della rete e mappatura delle fasi	34
3.2.2	Gestione dell'interverde	37
3.2.3	Dataset e generazione parametrica	37
3.2.4	Assunzioni e limiti del modello di simulazione	41
3.3	Baseline classiche	43
3.4	Architettura MetaSTGAT	44
3.4.1	State Encoder duale	45
3.4.2	Modulo spazio-temporale corrente (CST)	47
3.4.3	Modulo spaziale corrente (CS)	48
3.4.4	Testa Q-value	49
3.4.5	Meta-knowledge learner	49
3.4.6	Meta-GAT	52
3.4.7	Meta-LSTM	53
3.5	Differenze rispetto al modello di riferimento	55
3.6	Meccanismi spaziali alternativi	59
3.6.1	Meta-GCN	60
3.6.2	Meta-SONAR	61
3.7	Procedura di training e differenze dal paper originale	62
4	Confronto	65
4.1	Metriche di valutazione	65
4.2	Protocollo sperimentale	67
4.3	Confronti proposti	68
4.3.1	Studio del peso α	68
4.3.2	Studio di ablazione	68
4.3.3	Studio sui meccanismi spaziali	69
4.4	Risultati	69
4.4.1	Fixed-Time, MaxPressure e Pro: il confronto di base	70
4.4.2	Studio del peso α : risultati	71
4.4.3	Studio di ablazione: risultati	74
4.4.4	Studio sui meccanismi spaziali: risultati	76
4.4.5	Tabelle dei risultati.	78

Elenco delle figure

3.1	Rendering del visualizzatore di CITYFLOW per una singola intersezione a quattro vie: le indicazioni verde/rosso ai margini dell'incrocio mostrano i movimenti abilitati dalla fase corrente, i rettangoli colorati sulle corsie i veicoli in transito o in coda.	35
3.2	Rappresentazione grafica delle 8 fasi di Tabella 3.1: le frecce indicano i movimenti abilitati da ciascuna fase.	36
3.3	Rendering nel visualizzatore di CITYFLOW delle quattro combinazioni topologia-spaziatura di Tabella 3.2.	39
3.4	Schema a blocchi dell'architettura STGAT: dallo stato dell'intersezione s_i^t , attraverso l'encoder duale (MLP) e i moduli CS e CST (GAT e LSTM a pesi condivisi), fino all'azione a_i^t scelta dal DQN. Adattato da Wang et al. (2022).	46
3.5	Principio del meta-apprendimento applicato da MetaSTGAT: le meta-feature spaziali e temporali condizionano, tramite un meta-learner dedicato, i pesi dei layer LSTM e GAT usati per elaborare lo stato corrente del traffico. Adattato da Wang et al. (2022).	50
3.6	Il Meta-knowledge Learner: le meta-feature spaziali e temporali grezze vengono elaborate rispettivamente dallo SMK-Learner e dal TMK-Learner, producendo gli embedding $SMK(i)$ e $TMK(i)$ usati da Meta-GAT e Meta-LSTM. Adattato da Wang et al. (2022).	51
3.7	Meta-GAT: il modulo CS (sinistra) genera i pesi (W_j, b_j) della propria GAT a partire da SMK , il modulo CST (destra) genera (W_k, b_k) a partire da TMK ; entrambi producono l'output z_i del rispettivo modulo. Adattato da Wang et al. (2022).	54
3.8	Meta-LSTM "srotolata" nel tempo: a ogni istante t il meta-learner genera, da $TMK(i)$, i pesi W_Φ e b_Φ della cella LSTM che produce x_i^t dall'input e_i^t e dallo stato nascosto precedente h_i^{t-1}, c_i^{t-1} . Adattato da Wang et al. (2022).	55

3.9	Schema a blocchi completo di MetaSTGAT: rispetto a STGAT (Figura 3.4), GAT e LSTM sono sostituiti dalle rispettive versioni meta-condizionate, con i pesi generati dagli embedding $SMK(i)$ e $TMK(i)$ del Meta-knowledge Learner. Adattato da Wang et al. (2022).	56
4.1	Travel time medio (fascia scura), max wait (fascia intermedia) e travel time massimo (fascia chiara) per lo sweep su α , stessi dati delle Tabelle 4.4-4.5.	73
4.2	Variazione percentuale di travel time medio e massimo rispetto a MaxPressure (riferimento a 0%) per lo studio su α , calcolata per singola configurazione e poi mediata.	73
4.3	Travel time medio, max wait e travel time massimo sui preset di ablation, stessi dati delle Tabelle 4.6-4.7.	75
4.4	Variazione percentuale di travel time medio e massimo rispetto a MaxPressure (riferimento a 0%) sui preset di ablation, calcolata per singola configurazione/seed e poi mediata.	75
4.5	Travel time medio, max wait e travel time massimo sui meccanismi spaziali, stessi dati delle Tabelle 4.8-4.9.	79
4.6	Variazione percentuale di travel time medio e massimo rispetto a MaxPressure (riferimento a 0%) sui meccanismi spaziali, calcolata per singola configurazione/seed e poi mediata.	79

Elenco delle tabelle

3.1	Le 8 fasi semaforiche e i movimenti che ciascuna abilita.	36
3.2	Utilizzo di ciascuna combinazione topologia-spaziatura. Il dettaglio del confine tra training e validazione sulla griglia 4×4 a 100m è in Tabella 3.4.	38
3.3	Le cinque fasce del profilo di addestramento <i>complete</i> (totale: ~ 4.700 veicoli).	41
3.4	Tutte le combinazioni di griglia, spaziatura e profilo di traffico impiegate in questo lavoro, a meno delle varianti seed di validazione e test, con il relativo utilizzo e il nome del file di configurazione corrispondente.	42
4.1	Preset di ablazione e sottosistema isolato da ciascuno. La reward custom è definita da Eq. (3.3), mentre la reward paper è definita da Eq. (3.18).	69
4.2	Travel time medio, massimo, max wait e trip completion dei tre controllori presenti in ogni studio di questo capitolo, mediati sulle 3 configurazioni di training.	70
4.3	Come Tabella 4.2, mediata sulle 7 configurazioni di test.	70
4.4	Travel time medio, massimo e max wait, mediati sulle 3 configurazioni di training.	72
4.5	Come Tabella 4.4, mediata sulle 7 configurazioni di test.	72
4.6	Travel time medio, massimo e max wait sui preset di ablazione, mediati sulle 3 configurazioni di training.	74
4.7	Come Tabella 4.6, mediata sulle 7 configurazioni di test.	74
4.8	Travel time medio, massimo e max wait sui meccanismi spaziali, mediati sulle 3 configurazioni di training.	77
4.9	Come Tabella 4.8, mediata sulle 7 configurazioni di test.	77

4.10	Percentuale di trip completion tc per ogni modello e ogni configurazione, corretta escludendo dal denominatore i veicoli strutturalmente impossibilitati ad arrivare in tempo. Nessun modello scende sotto l'80%; a parte Fixed-Time (baseline non adattiva) l'unico caso ricorrente sotto il 90% è MetaSTGCN 1L.	80
4.11	Travel time medio $\overline{tt}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi.	81
4.12	Travel time massimo $tt_{\max}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi.	82
4.13	Max wait $wait_{\max}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi. Sulle config 4x4-200, valori a piena corsia.	83

Acronimi e Notazione

Acronimi

Acronimo	Significato
MDP	Markov Decision Process
RL	Reinforcement Learning
DQN	Deep Q-Network
PER	Prioritized Experience Replay
BPTT	Backpropagation Through Time
GNN	Graph Neural Network
GCN	Graph Convolutional Network
GAT	Graph Attention Network
TSC	Traffic Signal Control
SMK	Spatial Meta-Knowledge (learner)
TMK	Temporal Meta-Knowledge (learner)
CS	Cooperation Spatial (modulo Meta-GAT)
CST	Cooperation Spatial-Temporal (modulo Meta-GAT)

Notazione principale

Simbolo	Significato
$\mathcal{S}, \mathcal{A}, P, R, \gamma$	Stati, azioni, transizione, ricompensa, sconto
π, V^π, Q^π	Politica, funzione valore-stato, funzione valore-azione
θ, θ^-	Pesi della rete online e della target network
τ	Coefficiente di soft update
α (in RL)	Esponente di prioritizzazione del PER
α (in questo lavoro)	Peso della penalità anti-starvation nella ricompensa
ε	Probabilità di esplorazione
s_i^t, a_i^t, r_i^t	Stato, azione, ricompensa dell'intersezione i al tempo t
n_i^t, w_i^t, p_i^t	Veicoli per corsia, attesa massima per corsia, fase corrente
d_0	Dimensione dello stato in ingresso
d_1	Dimensione nascosta del modello
H, d_h	Numero di teste di attenzione e dimensione per testa
e_i^t, e_j^t	Embedding temporale e spaziale del Dual State Encoder
x_i^t, h_i^t, c_i^t	Output, stato nascosto e stato di cella del Meta-LSTM
$\text{SMK}(i), \text{TMK}(i)$	Meta-embedding spaziale e temporale dell'intersezione i
$z_{\text{CS}}(i), z_{\text{CST}}(i)$	Output dei moduli Meta-GAT spaziale e spazio-temporale
$\bar{tt}, tt_{\max}$	Travel time medio e massimo
tc	Trip completion, percentuale di veicoli che completano il percorso
$\text{wait}_{\max}$	Max wait, tempo massimo di un veicolo sulla stessa corsia senza superare l'incrocio

Capitolo 1

Introduzione

1.1 Motivazioni

Un incrocio semaforizzato stabilisce periodicamente quali flussi veicolari autorizzare e quali arrestare. Su una singola intersezione il problema è garantire che i veicoli in attesa continuino a scorrere in modo regolare. Le soluzioni classiche affrontano questo compito in due modi. Un piano a tempi fissi ripete una sequenza di fasi di durata prestabilita, calcolata una tantum su stime di traffico medie e mai più rivista in tempo reale. Un controllo attuato dai sensori concede invece il verde a una direzione solo quando un sensore rileva la presenza di un veicolo in coda, adattandosi così, entro certi limiti, alle variazioni del traffico osservato.

Se si considera una rete composta da più incroci, il problema cambia natura. La fase scelta a un'intersezione influenza, con un ritardo proporzionale alla distanza, il volume di veicoli in arrivo alle intersezioni adiacenti, per cui le decisioni prese nei singoli punti della rete non possono più essere indipendenti tra loro. Sistemi come SCOOT e SCATS rispondono a questa esigenza da diversi decenni, aggiustando periodicamente i piani semaforici di un'intera area urbana sulla base del traffico rilevato in tempo reale (Wei et al., 2020). Il controllo semaforico su rete si configura quindi come un problema di coordinamento distribuito, non di sola ottimizzazione isolata.

La diffusione di questi sistemi più adattivi nelle città resta comunque parziale, più per ostacoli economici e infrastrutturali che per assenza di alternative valide. Il problema del coordinamento semaforico resta però un banco di prova interessante a prescindere dalla sua adozione immediata sul campo, perché la stessa struttura, una rete di decisori locali le cui azioni si propagano ai nodi vicini con un ritardo temporale, ricorre in domini molto diversi dal traffico stradale. Lo stesso tipo di coordinamento serve, per esempio, a instradare i pacchi nei punti di smistamento di

un centro logistico, a far muovere senza collisioni una flotta di veicoli automatizzati in un magazzino, o a gestire un gruppo di ascensori in un edificio in modo che nessun piano resti scoperto troppo a lungo.

Valutare un controllore semaforico richiede di tenere conto di due proprietà distinte. La prima è l'efficienza, la capacità di ridurre il tempo che i veicoli impiegano ad attraversare la rete. La seconda è l'equità, la garanzia che nessuna direzione di traffico venga penalizzata sistematicamente a vantaggio delle altre. Le due proprietà non sono la stessa cosa e non migliorano necessariamente insieme. Un controllore può massimizzare il flusso complessivo e allo stesso tempo lasciare una minoranza di veicoli bloccata a lungo, se la sua logica di decisione non include alcuna attenzione esplicita al caso peggiore.

Tra i controllori reattivi allo stato dell'arte, MAX-PRESSURE (Varaiya, 2013) è tra i più studiati. È una regola locale che seleziona a ogni istante la fase capace di liberare più veicoli di quanti ne farebbe accumulare, calcolata dalla sola differenza tra i veicoli in coda sulle corsie in ingresso e in uscita che quella fase servirebbe. Il risultato teorico che lo rende celebre riguarda proprio l'orizzonte lungo, non solo la decisione istantanea. Se esiste una qualunque strategia capace di impedire che le code di una rete crescano senza limite nel tempo, MAX-PRESSURE la trova, pur usando solo informazione locale e senza conoscere in anticipo il volume di traffico in arrivo. Si tratta però di un risultato dimostrato sotto ipotesi teoriche idealizzate e verificato quasi soltanto in simulazione. Solo di recente alcuni progetti pilota hanno iniziato a testarne il funzionamento su hardware semaforico reale, e la sua diffusione resta comunque lontana da quella di sistemi commerciali consolidati come SCOOT e SCATS. La sua natura puramente greedy ha inoltre un costo sul fronte dell'equità. Un'intersezione posta su un'arteria ad alto scorrimento tende a favorire sistematicamente la direzione dominante, poiché la regola sceglie sempre la fase con il valore istantaneo più alto, senza mai considerare da quanto tempo attende la direzione esclusa. È una criticità che questo lavoro osserva empiricamente nel Capitolo 4.

Alcuni metodi recenti in letteratura affrontano il problema penalizzando esplicitamente le fasi responsabili di attese eccessive. FELight, per esempio, introduce una metrica di equità basata sui tempi di attesa dei veicoli e penalizza le fasi già identificate come responsabili di un ritardo superiore a una soglia prestabilita (Xiao et al., 2025). Questa tesi persegue lo stesso obiettivo con un meccanismo strutturalmente diverso, formalizzato nel Capitolo 3. Un termine dedicato della ricompensa penalizza l'attesa massima osservata su una corsia non servita dalla fase corrente, applicato a ogni singolo passo di decisione e per ogni intersezione, non soltanto quando un ri-

tardo supera una soglia o riguarda una fase individuata in anticipo. È il motivo per cui questo lavoro accetta, entro certi limiti, un tempo di percorrenza medio peggiore pur di ottenere un’attesa massima più contenuta. A parità di rete, un controllore che fa aspettare pochi veicoli molto a lungo non è preferibile a uno leggermente più lento in media ma che non lascia mai nessuno bloccato per un tempo indefinito.

Per verificare se questo meccanismo funzioni davvero, questo lavoro non si affida al solo segnale di ricompensa, che dipende dai propri stessi parametri e non è quindi un metro di giudizio imparziale, ma a una metrica puramente osservazionale calcolata identicamente per ogni controllore, indipendentemente da come è stato addestrato o se non è stato addestrato affatto. La metrica è il tempo massimo che un veicolo qualunque trascorre sulla stessa corsia senza superare l’incrocio, in una qualunque intersezione della rete, definita formalmente nel Capitolo 4.

L’obiettivo di questo lavoro è dunque duplice, e i suoi due termini non sono sempre allineati. Da un lato ridurre il tempo di percorrenza medio e massimo dei veicoli, l’**efficienza**. Dall’altro impedire che una direzione di traffico venga sistematicamente sacrificata a vantaggio delle altre, l’**equità**. Il Capitolo 4 mostra come questo compromesso si comporti concretamente al variare del peso assegnato alla componente di equità nella ricompensa.

A questi due obiettivi se ne affianca un terzo, di natura diversa dai primi due e altrettanto centrale in questo lavoro. Verificare che le prestazioni misurate non dipendano dalla specifica rete stradale su cui il modello è stato addestrato, ma si mantengano, almeno in parte, su topologie, spazature e condizioni di traffico mai incontrate durante il training. Efficienza ed equità sono gli obiettivi della politica di controllo. La **generalizzazione** è la condizione sotto cui questo lavoro pretende che entrambi restino validi.

Gran parte della letteratura recente sul controllo semaforico appreso non affronta sistematicamente questa domanda. La prassi più diffusa è addestrare e valutare un modello sulla stessa rete stradale, al più ripetendo l’intero esperimento, addestramento incluso, su una rete diversa, non trasferendo un singolo modello già addestrato a una rete che non ha mai visto (Wei et al., 2020). CoLight (Wei et al., 2019) conduce cinque esperimenti indipendenti, due su reti sintetiche e tre su reti reali, Manhattan, Hangzhou e Jinan, ciascuno con il proprio addestramento. MetaSTGAT (Wang et al., 2022), il modello di riferimento di questo lavoro, segue lo stesso schema su una griglia sintetica e due dataset reali. Quando in questi lavori compare il termine *generalization*, indica quasi sempre la tenuta a diverse intensità di traffico sulla stessa rete, non a una topologia strutturalmente diversa (Wang et al., 2026; Sun et al., 2026). Zhang et al. lo rendono esplicito nella motivazione di GeneraLight

(Zhang et al., 2020). La maggior parte dei modelli di reinforcement learning per il controllo semaforico è addestrata e valutata nello stesso ambiente di traffico, il che porta a overfitting e a una scarsa capacità di generalizzazione quando le condizioni reali cambiano.

Le eccezioni che esistono restano circoscritte a un'unica dimensione di variazione. AttendLight (Oroojlooy et al., 2020) addestra su un insieme eterogeneo di intersezioni singole e ne verifica le prestazioni su intersezioni mai viste, ma dichiara che l'estensione a una rete di più intersezioni coordinate resta lavoro futuro. Advanced-XLight (Zhang et al., 2022) misura un rapporto di trasferimento tra un modello addestrato su un dataset reale e valutato direttamente su un altro, senza ulteriore addestramento, ma sempre tra reti di scala paragonabile. Lo stesso GeneraLight generalizza al variare del flusso di traffico, non della topologia della rete.

Questo lavoro adotta invece la generalizzazione come requisito sistematico su assi che la letteratura esaminata non copre congiuntamente. Ogni modello, sia di reinforcement learning sia di riferimento classico, è addestrato una sola volta su una griglia 4×4 e valutato, senza alcun riaddestramento, su topologie di dimensione maggiore mai incontrate, 5×5 e 6×6 , su una spaziatura stradale diversa, e su profili di traffico differenti da quello di addestramento. Il Capitolo 4 descrive nel dettaglio il protocollo sperimentale adottato.

Il controllo appreso tramite reinforcement learning propone un paradigma diverso da quello di MAX-PRESSURE. Invece di una regola scritta a mano, una politica π viene addestrata a massimizzare una ricompensa che codifica l'obiettivo di rete, osservando ripetutamente le conseguenze delle proprie azioni in simulazione. Applicato a una singola intersezione, il problema è modellabile come un processo decisionale di Markov standard. Su scala di rete assume invece la forma di un sistema multi-agente a osservabilità parziale, dove ogni nodo rileva esclusivamente il proprio intorno locale, ed emerge quindi la necessità di un meccanismo per la condivisione delle informazioni tra agenti adiacenti. Le reti neurali a grafo assolvono a questa funzione. Rappresentando la rete stradale come un grafo, con le intersezioni come nodi e le strade come archi, l'informazione si propaga tra nodi vicini, permettendo a ogni intersezione di condizionare la propria decisione sullo stato dei suoi vicini senza richiedere una politica centralizzata.

MetaSTGAT (Wang et al., 2022) combina questi aspetti con uno ulteriore. Invece di condividere gli stessi pesi tra tutte le intersezioni della rete, un modulo di *meta-learning* genera dinamicamente i pesi di uno strato di attenzione e di uno strato ricorrente a partire da caratteristiche locali dell'intersezione, così che nodi diversi possano essere serviti da trasformazioni diverse pur restando un unico modello adde-

strato end-to-end. Gli autori originali riportano una riduzione del travel time medio rispetto a CoLight (Wei et al., 2019), un metodo di riferimento basato su *graph attention* ma con pesi condivisi. La combinazione di *attention* su grafo, ricorrenza e reinforcement learning non è isolata. Varianti di questo framework dominano la letteratura recente (Wang et al., 2026; Sun et al., 2026; Hu et al., 2026).

Fin qui si è discusso come il controllo semaforico su rete sia un problema di coordinamento distribuito, in cui l'efficienza aggregata non implica alcuna garanzia di equità tra le direzioni di traffico servite, come dimostra il caso di MAX-PRESSURE. I meccanismi di equità già proposti in letteratura intervengono in modo selettivo, solo oltre soglie o su fasi identificate in anticipo, mentre le architetture di controllo appreso capaci di coordinare intere reti di intersezioni, come MetaSTGAT, offrono il meccanismo di coordinamento ma non trattano l'equità come obiettivo esplicito di progettazione. La generalizzazione a topologie di rete, spaziatore stradali o profili di traffico mai osservati in addestramento resta infine raramente verificata in modo sistematico, anche quando il termine compare esplicitamente nei lavori citati. Questa tesi affronta congiuntamente questi tre aspetti, replicando da zero l'architettura e la procedura di addestramento di MetaSTGAT come base sperimentale.

1.2 Obiettivi e Contributi di questo lavoro

L'obiettivo principale di questo lavoro è verificare se un controllore appreso e coordinato tramite reti neurali a grafo possa garantire equità tra direzioni di traffico senza rinunciare all'efficienza raggiunta da un controllore puramente reattivo come MAX-PRESSURE, e se tale garanzia si mantenga su topologie, spaziatore e profili di traffico mai incontrati in addestramento. Un obiettivo secondario, strumentale al primo, è isolare quale parte dell'architettura e della procedura di addestramento sia effettivamente responsabile del comportamento osservato, invece di attribuirlo indistintamente all'insieme del modello. Nello specifico, i contributi di questa tesi sono i seguenti.

Un ambiente di simulazione multi-intersezione riproducibile. Questa tesi costruisce da zero un ambiente su CITYFLOW (Zhang et al., 2019), con reti stradali a griglia di dimensione 4×4 , 5×5 e 6×6 , due spaziatore stradali distinte, otto fasi semaforiche per intersezione e una formalizzazione del problema dettagliata nel Capitolo 3, insieme a una generazione parametrica dei dati di traffico con profili a intensità costante, a gradino e a giornata lavorativa. In ogni configurazione il traffico su una riga della griglia è potenziato artificialmente per simulare un'arteria

stradale, condizione necessaria per poter osservare uno squilibrio direzionale quando il traffico non è simmetrico.

Uno studio sistematico del compromesso tra efficienza ed equità. Variando il peso assegnato alla componente di equità nella ricompensa, questa tesi quantifica direttamente il compromesso che il meccanismo introduce, mostrando in particolare che rimuoverlo del tutto non produce un semplice guadagno di efficienza, ma un collasso della garanzia di equità verso il comportamento di MAX-PRESSURE, come mostra il Capitolo 4.

Uno studio di ablation sull'origine del comportamento osservato. Invece di presentare le differenze rispetto alla procedura originale come un insieme indistinto di miglioramenti, questa tesi isola singolarmente ognuna di esse, dal meccanismo di stato e ricompensa fino ai dettagli dell'algoritmo di apprendimento, per stabilire quale parte sia effettivamente necessaria alla garanzia di equità osservata e quale sia invece indipendente da essa.

Un confronto tra meccanismi di comunicazione spaziale a parità di resto dell'architettura. Questa tesi confronta tre meccanismi di comunicazione tra intersezioni, *graph attention*, convoluzione grafica e propagazione a onda ispirata a SONAR, mantenendo identico il resto dell'architettura, per verificare se il comportamento osservato dipenda specificamente dal meccanismo di attenzione appreso o sia una proprietà più generale della coordinazione locale via rete neurale a grafo.

Un protocollo di valutazione a generalizzazione sistematica. Ogni modello di questa tesi, appreso o classico, è valutato senza riaddestramento su topologie, spazature e profili di traffico mai incontrati in addestramento, riportando media e caso peggiore del tempo di viaggio, la percentuale di veicoli che completano il proprio percorso e la massima attesa registrata da un veicolo allo stesso incrocio.

1.3 Struttura della tesi

Il Capitolo 2 introduce il background necessario a leggere i capitoli successivi senza assumere familiarità pregressa con nessuno dei campi coinvolti, dal reinforcement learning alle reti neurali ricorrenti, dalle reti neurali a grafo al controllo semaforico. Il Capitolo 3 formalizza il problema affrontato, descrive l'ambiente di simulazione e le baseline classiche, e presenta l'architettura di MetaSTGAT e le sue varianti insieme a ogni differenza introdotta rispetto all'articolo originale, motivata caso per

caso. Il Capitolo 4 descrive il protocollo sperimentale e riporta i risultati ottenuti sui tre studi, peso dell'equità, ablation, meccanismi spaziali. Il Capitolo 5 discute i risultati alla luce dei tre obiettivi dichiarati in questa introduzione, indica le direzioni lasciate aperte e dichiara i limiti dello studio.

Capitolo 2

Background

Questo capitolo introduce, da zero, ogni concetto usato nel resto della tesi, così da poter leggere i Capitoli 3 e 4 senza assumere familiarità pregressa con nessuno dei campi coinvolti. I quattro strumenti generali discussi (reti neurali, apprendimento per rinforzo, reti neurali ricorrenti, grafi e reti neurali a grafo) sono sviluppati in letteratura per problemi che non hanno nulla a che vedere con il traffico: la loro applicazione a questo problema specifico è discussa solo nell'ultima sezione del capitolo e formalizzata nel dettaglio nel Capitolo 3.

2.1 Struttura del capitolo

Il capitolo comprende cinque sezioni. Questa sezione ne anticipa il contenuto, motivando anche perché alcuni argomenti correlati non vengano discussi, così da poter leggere il resto del capitolo senza doverne indovinare i confini.

Nella §2.2 si introducono i concetti minimi di rete neurale (perceptrone, strati, funzione di loss, discesa del gradiente, backpropagation) su cui ogni rete usata in questo lavoro, dal DQN alla LSTM alle reti a grafo, è costruita. Non si discute l'apprendimento supervisionato in profondità, né alcuna architettura per dati a griglia regolare come le *convolutional neural network*: lo stato osservato da un'intersezione in questo lavoro è un vettore di conteggi e tempi di attesa per corsia, non un'immagine, per cui l'elaborazione di immagini non è mai rilevante.

Nella §2.3 si introduce il reinforcement learning, dal paradigma agente-ambiente fino alle estensioni del DQN effettivamente usate in questo lavoro, passando per la distinzione tra apprendimento model-based e model-free e per l'apprendimento multi-agente. Il reinforcement learning model-based, pur introdotto per completezza definitoria, non è discusso oltre la sua definizione: nessun modello usato in questa tesi apprende o usa un modello esplicito della dinamica dell'ambiente (§2.3.3).

Nella §2.4 si introduce la necessità di una memoria per dati sequenziali e la cella LSTM che questo lavoro adotta per integrare l'informazione temporale. Si discute brevemente anche la GRU, un'alternativa più leggera, solo per contrasto: questo lavoro non la utilizza.

Nella §2.5 si introduce il concetto di grafo da zero e il *message passing* su cui ogni *graph neural network* è basata, con particolare attenzione alla GCN e alla GAT, le due formulazioni confrontate sperimentalmente nel Capitolo 4 tramite le rispettive varianti meta-condizionate. Meccanismi di comunicazione multi-agente alternativi alle GNN, come i protocolli di comunicazione appresi esplicitamente tra agenti (non basati su un grafo di adiacenza), esulano dallo scopo di questo lavoro, che adotta sistematicamente la comunicazione locale via GNN discussa in questa sezione, e non vengono quindi trattati.

Infine, nella §2.6 le quattro sezioni precedenti confluiscono nel problema applicativo di questo lavoro: si introduce il vocabolario di base del controllo semaforico, le tre generazioni di metodi di controllo (a tempi fissi, reattivo, appreso) e si posiziona esplicitamente MetaSTGAT, il modello di riferimento di questo lavoro, all'interno di questa tassonomia.

2.2 Reti Neurali

Una **rete neurale** è una funzione parametrica che trasforma un input in un output attraverso una successione di trasformazioni semplici, i cui parametri (i **pesi**) vengono appresi dai dati invece di essere scritti a mano. L'unità di base è il **neurone artificiale** (o *perceptrone*): riceve un vettore di input $x \in \mathbb{R}^n$, ne calcola una combinazione lineare pesata $z = w^\top x + b$ (con w vettore di pesi e b un termine costante detto *bias*), e applica a z una **funzione di attivazione** non lineare σ , producendo l'output $a = \sigma(z)$. Attivazioni comuni sono la sigmoide $\sigma(z) = 1/(1 + e^{-z})$, che comprime z in $(0, 1)$ ed è quindi utile ovunque serva un valore interpretabile come probabilità o come "quanto lasciar passare" (i *gate* dell'LSTM, §2.4, ne sono un esempio), la tangente iperbolica $\tanh$, che lo comprime invece in $(-1, 1)$, e la *ReLU* $\max(0, z)$, che si limita ad azzerare i valori negativi. La non linearità è essenziale: senza di essa, comporre più trasformazioni lineari produrrebbe comunque una singola trasformazione lineare, vanificando il vantaggio di avere più strati.

Una rete *feedforward* (o *multi-layer perceptron*, MLP) dispone i neuroni in **strati** (*layer*) successivi: l'output di ogni strato è l'input dello strato seguente, dall'*input layer* (i dati grezzi) attraverso uno o più *hidden layer* fino all'*output layer* finale. In forma matriciale, un singolo strato calcola $h^{(l+1)} = \sigma(W^{(l)}h^{(l)} + b^{(l)})$, dove $h^{(l)}$

raccoglie gli output di tutti i neuroni dello strato l e $W^{(l)}$ è la matrice che impila i pesi di ciascun neurone di quello strato. Impilare più strati permette alla rete di comporre trasformazioni via via più astratte dell'input originale.

Addestramento: discesa del gradiente e backpropagation. Una rete neurale non nasce già utile: i suoi pesi partono da valori casuali e vengono corretti iterativamente confrontando l'output prodotto con quello desiderato. Una **funzione di loss** $\mathcal{L}(\hat{y}, y)$ quantifica la distanza tra l'output prodotto $\hat{y}$ e quello desiderato y (o, nel caso del reinforcement learning discusso a seguire, tra una stima e un target calcolato da altre stime). L'addestramento consiste nel regolare ogni peso nella direzione che riduce $\mathcal{L}$, un'operazione nota come **discesa del gradiente**:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w},$$

dove η , il **tasso di apprendimento** (*learning rate*), controlla l'ampiezza di ogni aggiornamento. Calcolare la derivata parziale $\partial \mathcal{L} / \partial w$ per ogni peso della rete, dal più vicino all'output al più lontano, richiede applicare ripetutamente la regola della catena del calcolo differenziale attraverso tutti gli strati intermedi: l'algoritmo che esegue questo calcolo in modo efficiente, una sola passata dall'output verso l'input, è la **backpropagation**. Nella pratica, il gradiente si stima su un piccolo sottoinsieme casuale di dati (*mini-batch*) alla volta invece che sull'intero insieme di dati disponibile, una variante nota come discesa del gradiente stocastica, molto più economica computazionalmente a fronte di una stima del gradiente più rumorosa ma sufficiente a guidare comunque l'addestramento verso una soluzione utile.

Oltre la discesa del gradiente pura. La regola di aggiornamento vista sopra usa lo stesso tasso di apprendimento η per ogni peso della rete, un limite quando la superficie di loss ha curvatura molto diversa da un peso all'altro (alcune direzioni richiederebbero un passo grande, altre uno minimo): un ottimizzatore più sofisticato adatta l'ampiezza dell'aggiornamento parametro per parametro, sulla base della storia recente dei gradienti osservati. Il **momentum** accumula una media mobile esponenziale dei gradienti passati e aggiorna i pesi in quella direzione media invece che nella sola direzione del gradiente corrente, smorzando le oscillazioni lungo le direzioni in cui il segno del gradiente cambia di continuo. **RMSprop** (Tieleman and Hinton, 2012) adotta un principio complementare: mantiene per ciascun peso una media mobile del quadrato dei gradienti recenti e divide l'aggiornamento per la sua radice, riducendo il passo sui pesi il cui gradiente è stato storicamente grande in valore assoluto e aumentandolo su quelli il cui gradiente è stato piccolo, così da

rendere il progresso più uniforme tra pesi con scale molto diverse. **Adam** (Kingma and Ba, 2015) combina entrambe le idee (media mobile del gradiente e del suo quadrato) ed è oggi l’ottimizzatore di riferimento per la maggior parte delle reti neurali profonde. **Questo lavoro adotta RMSprop**, coerentemente con il modello di riferimento (Wang et al., 2022), per l’addestramento di ogni rete Q descritta nel Capitolo 3.

Ogni rete usata in questo lavoro, dal DQN (§2.3) alla LSTM (§2.4) alle reti a grafo (§2.5), è in ultima analisi una particolare composizione di questi stessi due ingredienti: strati di neuroni con un’attivazione non lineare, addestrati per discesa del gradiente tramite backpropagation. Ciò che cambia da un’architettura all’altra è come gli strati sono collegati tra loro, non il principio di addestramento sottostante.

2.3 Reinforcement Learning

2.3.1 Il paradigma agente-ambiente

Il *reinforcement learning* (RL, apprendimento per rinforzo) è un paradigma di apprendimento automatico in cui un **agente** impara a comportarsi interagendo ripetutamente con un **ambiente**, senza che nessuno gli mostri esempi di comportamento corretto. A ogni passo di tempo t , l’agente osserva lo stato corrente dell’ambiente, sceglie un’**azione** tra quelle disponibili, e riceve in cambio due cose: un nuovo stato (l’ambiente è cambiato, anche per effetto dell’azione scelta) e un numero scalare, la **ricompensa**, che indica quanto quell’azione sia stata desiderabile in quel momento. Non esiste alcun supervisore che dica all’agente qual è l’azione giusta da compiere: l’unico segnale disponibile è la ricompensa, ottenuta *dopo* aver agito, e spesso in ritardo rispetto alla decisione che l’ha causata (una fase semaforica scelta ora può congestionare un incrocio vicino solo dopo diversi passi). Il problema che il RL affronta è per l’appunto questo: imparare, per tentativi ripetuti, quale comportamento massimizzi la somma delle ricompense ricevute nel tempo, non solo quella immediata.

Questa formulazione è deliberatamente generale: lo stesso paradigma descrive un programma che impara a giocare a scacchi, un braccio robotico che impara ad afferrare un oggetto, e, come in questa tesi, un semaforo che impara a scegliere quale fase attivare. Ciò che cambia da un problema all’altro sono solo le definizioni concrete di stato, azione e ricompensa.

Vale la pena distinguere questo paradigma dall’**apprendimento supervisionato**, il paradigma di machine learning più diffuso e spesso il primo che si incontra: lì una rete neurale (§2.2) riceve coppie input-output già corrette (ad esempio migliaia

di immagini, ciascuna già etichettata con il nome dell'oggetto che raffigura) e impara a riprodurre quella corrispondenza. Nel RL non esiste alcuna etichetta corretta da imitare: l'agente deve scoprire da solo, per tentativi, quale azione sia buona in ciascuno stato, guidato solo da un segnale scalare di ricompensa che oltretutto può arrivare con ritardo rispetto alla decisione che l'ha causata. È un problema strutturalmente più difficile, ma anche l'unico adatto a situazioni, come il controllo semaforico, in cui nessuno conosce a priori quale sia la "decisione corretta" da etichettare: lo si può solo osservare indirettamente dalle sue conseguenze sul traffico.

2.3.2 Processo decisionale di Markov

La formalizzazione matematica standard di un problema di RL è il **processo decisionale di Markov** (MDP) (Sutton and Barto, 2018), definito dalla tupla

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

dove $\mathcal{S}$ è l'insieme di tutti gli stati possibili dell'ambiente, $\mathcal{A}$ l'insieme di tutte le azioni possibili, $P(s' | s, a)$ la probabilità di transitare nello stato s' avendo eseguito l'azione a nello stato s , $R(s, a)$ la ricompensa attesa per quella stessa coppia, e $\gamma \in [0, 1)$ il **fattore di sconto**, che pondera l'importanza delle ricompense future rispetto a quelle immediate: un γ vicino a 0 rende l'agente "miope", interessato quasi solo alla ricompensa immediata; un γ vicino a 1 lo rende attento anche a conseguenze lontane nel tempo. Il nome "Markov" indica un'assunzione precisa: la probabilità di transizione $P(s' | s, a)$ dipende solo dallo stato corrente s , mai dalla sequenza di stati che ha portato fin lì, cosa che richiede che lo stato s contenga già tutta l'informazione rilevante per decidere.

Un agente segue una **politica** $\pi(a | s)$, una distribuzione di probabilità sulle azioni condizionata allo stato corrente; il suo obiettivo è massimizzare il **ritorno** atteso

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

ovvero la somma scontata delle ricompense future a partire dall'istante t . Per ragionare su quanto sia buona una politica prima ancora di eseguirla fino in fondo, si definiscono due funzioni valore: la funzione valore-stato $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$ e la funzione valore-azione $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$, che misurano rispettivamente quanto ritorno atteso si ottiene seguendo π da uno stato, o da una coppia stato-azione. Queste soddisfano l'**equazione di Bellman**, che esprime il valore di uno stato in funzione del valore atteso degli stati successivi, invece che dover

simulare l'intero futuro:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a) + \gamma V^\pi(s')] \quad (2.1)$$

Per una politica ottima π^* che massimizza $V^\pi(s)$ per ogni stato simultaneamente, vale l'analoga equazione di Bellman di ottimalità:

$$Q^*(s, a) = \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[R(s, a) + \gamma \max_{a'} Q^*(s', a') \right] \quad (2.2)$$

Se si conoscesse Q^* , la politica ottima sarebbe immediata da ricavare: scegliere sempre $\operatorname{argmax}_a Q^*(s, a)$. Il problema del RL si riduce quindi, in buona parte, a stimare Q^* senza conoscerlo a priori.

2.3.3 Apprendimento model-based e model-free

Esiste una distinzione fondamentale tra due famiglie di algoritmi RL, a seconda che l'agente costruisca o meno una rappresentazione esplicita della dinamica dell'ambiente. Un algoritmo **model-based** apprende (o riceve) un modello approssimato di $P(s' | s, a)$ e $R(s, a)$, e lo usa per pianificare, cioè per simulare mentalmente le conseguenze di più sequenze di azioni prima di sceglierne una, senza dover necessariamente eseguirle davvero nell'ambiente. Un algoritmo **model-free**, al contrario, non costruisce alcun modello della dinamica: stima direttamente Q^* (o la politica) dall'esperienza osservata, senza mai provare a prevedere esplicitamente lo stato successivo. Il vantaggio del model-free è la semplicità e la minore dipendenza dalla qualità di un modello che potrebbe essere difficile da apprendere accuratamente; lo svantaggio è un uso tipicamente meno efficiente dell'esperienza raccolta, poiché non la si riusa per simulare scenari mai osservati direttamente.

Questo lavoro adotta un approccio interamente model-free: né il modello di riferimento (Wang et al., 2022) né le varianti implementate in questa tesi (Capitolo 3) apprendono o usano un modello esplicito della dinamica del traffico. L'agente stima direttamente una funzione Q dall'esperienza simulata in CityFlow, con la famiglia di tecniche descritta nel resto di questa sezione.

2.3.4 Q-learning e Deep Q-Network

Il **Q-learning** tabellare approssima Q^* applicando iterativamente l'Equazione 2.2 come regola di aggiornamento, aggiustando il valore stimato $Q(s, a)$ verso il target $r + \gamma \max_{a'} Q(s', a')$ osservato a ogni transizione. Il nome "tabellare" indica che Q è letteralmente una tabella con una riga per ogni stato e una colonna per ogni

azione: un approccio che richiede una tabella con tutte le coppie (s, a) , il che risulta inapplicabile per spazi di stato $\mathcal{S}$ continui o ad alta dimensionalità, come nel caso del controllo semaforico, dove lo stato include conteggi di veicoli e tempi di attesa che assumono innumerevoli combinazioni possibili (Li, 2018).

Un esempio numerico. Si supponga un singolo incrocio con due sole azioni possibili, "verde a Nord-Sud" (a_1) e "verde a Est-Ovest" (a_2), $\gamma = 0.9$, e una tabella Q ancora azzerata ovunque. In un episodio di addestramento, dallo stato s (coda presente su entrambe le direzioni) l'agente esegue a_1 per esplorazione casuale (ε -greedy, discusso a breve), osserva una ricompensa $r = -2$ (qualche veicolo è comunque rimasto in coda a Est-Ovest) e transita in un nuovo stato s' in cui $\max_{a'} Q(s', a')$ vale, poniamo, 5 (una stima già presente da esperienza precedente). L'aggiornamento tabellare del Q-learning corregge la stima verso il target di Bellman con un tasso η :

$$Q(s, a_1) \leftarrow Q(s, a_1) + \eta \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{target} = -2 + 0.9 \cdot 5 = 2.5} - Q(s, a_1) \right] = 0 + \eta(2.5 - 0) = 2.5 \eta.$$

Ripetendo questo aggiornamento su molte transizioni campionate, la stima $Q(s, a_1)$ converge verso il vero ritorno atteso dell'eseguire a_1 in s e poi seguire la politica ottima in seguito, senza che nessuno abbia mai detto esplicitamente all'agente se a_1 fosse la scelta giusta in s : l'informazione emerge indirettamente dalla combinazione di ricompensa osservata e stima del valore dello stato successivo. Il DQN, descritto di seguito, sostituisce la tabella con una rete neurale ma applica esattamente lo stesso principio di aggiornamento verso il target di Bellman.

Deep Q-Network (DQN). Mnih et al. (2015) superano tale limite sostituendo la tabella con una rete neurale $Q(s, a; \theta)$, che generalizza a stati mai visti esattamente interpolando tra stati simili osservati durante l'addestramento, i cui parametri (pesi) θ vengono aggiornati per discesa del gradiente in modo che $Q(s, a; \theta)$ si avvicini al target di Bellman. Per mitigare l'instabilità dell'addestramento dovuta alla correlazione temporale dei campioni (stati consecutivi di uno stesso episodio sono molto simili tra loro, violando l'assunzione di campioni indipendenti richiesta dalla discesa del gradiente stocastico), gli autori introducono inoltre il *replay buffer*: una memoria in cui si accumulano le transizioni (s, a, r, s') osservate, da cui si campiona poi mini-batch in ordine casuale per l'addestramento, rompendo la correlazione temporale.

2.3.5 Esplorazione ε -greedy

Un agente che seguisse sempre e solo l'azione a valore Q stimato più alto non raccoglierebbe mai esperienza sulle conseguenze di azioni alternative, rischiando di convergere su una politica sub-ottima per via di una mancata esplorazione: se per puro caso, nelle prime fasi dell'addestramento, un'azione appare leggermente migliore di un'altra, un agente completamente "goloso" (*greedy*) non la riprova mai più, e non scopre mai se l'altra fosse in realtà preferibile. La strategia più comune per bilanciare sfruttamento (*exploitation*, scegliere l'azione ritenuta migliore) ed esplorazione (*exploration*, raccogliere informazione su azioni meno note) è ε -greedy: a ogni passo di decisione, con probabilità ε l'agente sceglie un'azione uniformemente a caso, e con probabilità $1 - \varepsilon$ sceglie $\arg\max_a Q(s, a; \theta)$. Il valore di ε tipicamente decresce nel corso dell'addestramento, da un valore iniziale vicino a 1 (quasi solo esplorazione, quando la stima di Q è ancora inaffidabile) verso un valore finale piccolo ma non nullo (quasi solo sfruttamento, mantenendo comunque un minimo di esplorazione residua).

2.3.6 Estensioni usate in questo lavoro

Il DQN di base ammette diverse estensioni indipendenti, ciascuna delle quali corregge una specifica fonte di instabilità o inefficienza; la procedura di training descritta nel Capitolo 3 le adotta tutte, motivandole caso per caso rispetto all'articolo di riferimento:

- Il **Double DQN** (van Hasselt et al., 2016) disaccoppia la selezione dell'azione dalla sua valutazione, usando la rete online per scegliere l'azione migliore e una *target network* $Q(\cdot; \theta^-)$, una copia periodicamente sincronizzata della rete online, solo per valutarla: $y = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-)$. Questo meccanismo evita il problema del *moving target*, che si verifica ottimizzando una stima rispetto a parametri in continua variazione, e riduce il bias sistematico di sovrastima del DQN standard (usare la stessa rete sia per scegliere sia per valutare l'azione tende a preferire azioni il cui valore è per caso sovrastimato dal rumore di stima).
- Il **Prioritized Experience Replay** (PER, Schaul et al., 2016) sostituisce il campionamento uniforme dal replay buffer con un campionamento proporzionale all'errore di differenza temporale (*TD error*) δ_i , la discrepanza tra il valore stimato e il target di Bellman per quella transizione: le transizioni che la rete predice peggio vengono ricampionate più spesso, concentrando la capacità di apprendimento dove serve di più. Assegnando a ciascuna transizione

una priorità $p_i = |\delta_i| + \varepsilon$, la probabilità di campionamento è $P(i) = p_i^\alpha / \sum_k p_k^\alpha$, con $\alpha \in [0, 1]$ iperparametro che ne controlla la dipendenza dall'errore. Per correggere il bias introdotto da un campionamento non uniforme, la loss viene pesata tramite coefficienti di *importance sampling* $w_i = (N \cdot P(i))^{-\beta}$, dove N è la dimensione del buffer e $\beta \in [0, 1]$ iperparametro dinamico che ne controlla il grado di correzione.

- La **Huber loss** (o *smooth L1*) sostituisce l'errore quadratico medio (MSE) con una funzione quadratica vicino allo zero e lineare lontano da esso, risultando meno sensibile agli outlier prodotti da transizioni con errore di stima molto grande (con MSE, un singolo errore enorme domina il gradiente dell'intero batch; con Huber il suo contributo cresce solo linearmente):

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{se } |y - \hat{y}| \leq \delta \\ \delta \left(|y - \hat{y}| - \frac{1}{2}\delta \right) & \text{altrimenti} \end{cases} \quad (2.3)$$

Tipicamente $\delta = 1$.

- Il **soft update** (*Polyak averaging*) sostituisce l'aggiornamento periodico discreto della *target network* con una media mobile esponenziale: $\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$ con $\tau \ll 1$, stabilizzando le variazioni del target a ogni passo di ottimizzazione invece di farlo saltare bruscamente ogni volta che la copia periodica scatta.
- Quando la rete Q include un modulo ricorrente (§2.4), l'assunzione di Markov sulle singole transizioni campionate dal replay buffer viene invalidata: lo stato ricorrente (h, c) usato durante il training viene inizializzato a zero, introducendo un disallineamento rispetto allo stato generato sequenzialmente durante la raccolta dati, un problema noto come *hidden state staleness*. Seguendo Kapturowski et al. (2019), si campiano intere sequenze di lunghezza L e si dedicano i primi passi della sequenza (*burn-in*) esclusivamente a ricostruire uno stato ricorrente plausibile, senza calcolare gradiente, prima di eseguire la *backpropagation through time* (BPTT, §2.4) sui passi restanti.

2.3.7 Apprendimento multi-agente

Tutto quanto descritto finora assume un solo agente che interagisce con un solo ambiente. Quando più agenti operano nello stesso ambiente, ciascuno osservando solo una porzione dello stato globale e influenzando con le proprie azioni anche ciò

che gli altri agenti osservano e ricevono come ricompensa, il problema si generalizza da MDP a **processo decisionale di Markov decentralizzato e parzialmente osservabile** (Dec-POMDP): "decentralizzato" perché non esiste un'unica azione congiunta scelta da un solo decisore, ma N azioni scelte indipendentemente da N agenti; "parzialmente osservabile" perché ogni agente vede solo il proprio stato locale, non lo stato globale completo dell'ambiente. Quest'ultima proprietà non è solo teorica: in questo lavoro si concretizza nel campo visivo limitato di ogni intersezione (Capitolo 3), che esclude deliberatamente dallo stato osservato ciò che accade oltre una certa distanza dalla corsia, ben prima che la questione multi-agente entri in gioco.

Il multi-agente introduce una difficoltà che il caso a singolo agente non ha: la **non stazionarietà**. L'equazione di Bellman e l'intera teoria del Q-learning assumono che l'ambiente risponda in modo coerente nel tempo alla stessa azione nello stesso stato; ma se più agenti apprendono simultaneamente, ciascuno sta anche cambiando il proprio comportamento mentre gli altri lo osservano, cosicché per ogni singolo agente l'ambiente (che include il comportamento degli altri agenti) appare cambiare regole nel tempo, anche se le vere dinamiche fisiche sottostanti restano identiche. Nessuna delle garanzie di convergenza valide per il RL a singolo agente si applica quindi automaticamente al caso multi-agente.

Una seconda difficoltà, distinta dalla non stazionarietà, è l'**assegnazione del merito** (*credit assignment*): quando più agenti contribuiscono a un solo risultato osservato, non è ovvio quale frazione di quel risultato attribuire a ciascuno. Il caso classico è una singola ricompensa globale condivisa da tutti gli agenti: un agente non ha modo diretto di distinguere quanto di quella ricompensa dipenda dalla propria azione e quanto dalle azioni altrui, un problema che soluzioni come COMA (Li, 2018) affrontano stimando un termine controfattuale, quanto sarebbe cambiata la ricompensa se il singolo agente avesse agito diversamente a parità di comportamento altrui. Il problema di questo lavoro è più mite: ogni intersezione riceve una ricompensa r_i^t calcolata esclusivamente sul proprio stato locale (Capitolo 3), non una ricompensa globale condivisa, evitando così la versione più severa del problema appena descritto. Una versione attenuata resta comunque presente: lo stato locale di un'intersezione, e quindi la sua stessa ricompensa, dipende in parte dalle code di traffico generate dalle scelte delle intersezioni a monte, per cui l'effetto della propria azione e quello dell'azione altrui restano in parte intrecciati anche con una ricompensa puramente locale.

Due schemi di coordinamento ricorrono in letteratura (Li, 2018): l'*addestramento centralizzato con esecuzione decentralizzata* (CTDE), in cui gli agenti condivide-

no informazione globale solo durante il training per poi decidere localmente, e l'*addestramento ed esecuzione entrambi decentralizzati* (DTDE), in cui l'informazione resta sempre e solo locale, anche durante l'addestramento. Una scelta indipendente da questa è se i diversi agenti condividano o meno gli stessi pesi (*parameter sharing*): dividerli riduce il numero di parametri totali e permette a un agente di beneficiare dell'esperienza raccolta da tutti gli altri, a costo di una minore capacità di specializzarsi su condizioni locali peculiari, a meno che il meccanismo non trovi un altro modo per differenziare il comportamento tra agenti pur condividendo i pesi.

Questo lavoro è un sistema multi-agente: ogni intersezione della rete stradale è un agente a sé, con un proprio stato s_i^t , una propria azione a_i^t (la fase semaforica scelta) e una propria ricompensa r_i^t , calcolati indipendentemente per ogni intersezione i (Capitolo 3); è per questo che il titolo di questa tesi parla esplicitamente di apprendimento per rinforzo *multi-agente*. L'architettura è di tipo DTDE, con scambio di informazione locale tra intersezioni vicine (§2.5), e adotta il parameter sharing: tutte le intersezioni condividono la stessa rete Q , ma un meccanismo di meta-learning (§2.5) genera pesi diversi per nodi con caratteristiche locali diverse, così da attenuare proprio il costo della condivisione appena descritto.

2.4 Reti Neurali Ricorrenti

Una rete neurale *feedforward* calcola il proprio output come funzione del solo input corrente: ogni esempio è trattato in modo indipendente dai precedenti, e la rete non conserva alcuna memoria di cosa abbia visto in passato. Questa assunzione fallisce ogni volta che i dati hanno una struttura sequenziale in cui l'ordine e la storia contano quanto il singolo valore osservato (testo, audio, serie temporali, o lo stato di un'intersezione stradale nei passi decisionali precedenti) (Lipton et al., 2015). Le *reti neurali ricorrenti* (RNN) affrontano il problema introducendo un ciclo: a ogni istante t , un nodo riceve in ingresso non solo il dato corrente $x^{(t)}$ ma anche il proprio stato nascosto al passo precedente $h^{(t-1)}$,

$$h^{(t)} = \sigma(W^{hx}x^{(t)} + W^{hh}h^{(t-1)} + b_h),$$

così che l'informazione rilevante di tutta la sequenza osservata fino a t possa, in linea di principio, persistere in $h^{(t)}$: lo stesso insieme di pesi W^{hx} , W^{hh} , b_h viene riapplicato a ogni istante di tempo, invece di avere pesi diversi per ogni passo. "Srotolando" la rete lungo l'asse del tempo, cioè disegnando esplicitamente una copia dei nodi per ogni istante $t = 1, 2, \dots$ collegata alla successiva, ogni passo diventa uno strato di una rete feedforward molto profonda a pesi condivisi, addestrabile con la stessa

backpropagation vista in precedenza applicata lungo la sequenza anziché lungo i layer: questo algoritmo prende il nome di *backpropagation through time* (BPTT).

Il problema del gradiente. Una rete ricorrente addestrata su sequenze lunghe soffre di un problema strutturale: il gradiente retropropagato attraverso molti passi temporali tende a annullarsi (*vanishing gradient*) o a divergere (*exploding gradient*) a seconda che i pesi ricorrenti abbiano modulo minore o maggiore di uno, poiché il gradiente si ottiene moltiplicando ripetutamente per lo stesso fattore a ogni passo srotolato, esattamente come una potenza c^n tende a zero o esplode al crescere di n a seconda che $|c|$ sia minore o maggiore di uno. Nella pratica, questo rende di fatto impossibile apprendere dipendenze che coprono più di poche decine di passi con una RNN semplice.

Long Short-Term Memory (LSTM). L'LSTM (Lipton et al., 2015) risolve il problema sostituendo il semplice nodo ricorrente con una *cella di memoria*: uno stato interno $c^{(t)}$ collegato a se stesso da un percorso a peso fisso unitario (il *constant error carousel*), attraverso cui il gradiente può fluire per molti passi senza annullarsi né esplodere, non dovendo più moltiplicare ripetutamente per un peso appreso potenzialmente minore o maggiore di uno. L'accesso a questo stato è regolato da tre *gate*, unità sigmoidali il cui valore in $[0, 1]$ modula quanta informazione passi attraverso di esse, in modo analogo a una valvola che si apre o si chiude in proporzione al segnale che riceve. Con pesi $W_g \in \mathbb{R}^{2d_1 \times d_1}$ e bias $b_g \in \mathbb{R}^{d_1}$ condivisi da ogni nodo, per ciascuno dei quattro gate $g \in \{\text{forget, input, output, cell}\}$:

$$\begin{bmatrix} f_i^t \\ l_i^t \\ o_i^t \\ g_i^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(W_{\{f, l, o, g\}}^\top [e_i^t \parallel h_i^{t-1}] + b_{\{f, l, o, g\}} \right), \quad (2.4)$$

$$c_i^t = f_i^t \odot c_i^{t-1} + l_i^t \odot g_i^t,$$

$$x_i^t = h_i^t = o_i^t \odot \tanh(c_i^t);$$

il *forget gate* f decide quanto dello stato precedente dimenticare, l'*input gate* l quanto del nuovo contenuto candidato g (talvolta indicato $\tilde{c}$ in altre notazioni) incorporare nello stato aggiornato, e l'*output gate* o quanto dello stato risultante esporre come output della cella. Un valore di forget gate vicino a 1 lascia passare l'informazione pregressa quasi inalterata anche dopo molti passi; uno vicino a 0 la cancella, permettendo alla rete di "dimenticare" deliberatamente informazione non più rilevante. La

notazione con pedice i e apice t , anziché quella generica appena introdotta, anticipa l'uso di questa stessa equazione (Equazione 3.7) nel Capitolo 3 per il modulo ricorrente di MetaSTGAT, dove i indicizza l'intersezione ed e_i^t sostituisce l'input generico $x^{(t)}$: ogni intersezione mantiene quindi una propria cella di memoria, aggiornata a ogni passo decisionale con il proprio stato osservato.

Gated Recurrent Unit (GRU). Una variante più leggera dell'LSTM, la **GRU** (Cho et al., 2014), fonde il *forget gate* e l'*input gate* in un unico *update gate* e rinuncia allo stato di cella $c^{(t)}$ separato, esponendo un solo stato nascosto $h^{(t)}$ sia come memoria interna sia come output: meno parametri da addestrare e un grafo computazionale più semplice, a fronte di una capacità di rappresentazione leggermente inferiore rispetto ai quattro gate distinti dell'LSTM. Le due varianti raggiungono nella pratica prestazioni simili su molti compiti, e la scelta tra loro è spesso guidata più dalla convenzione di un'architettura di riferimento che da un vantaggio netto dell'una sull'altra: questo lavoro adotta l'LSTM per coerenza con il modello di riferimento (Wang et al., 2022).

2.5 Grafi e Graph Neural Network

2.5.1 Cos'è un grafo

Un **grafo** $G = (V, E)$ è una struttura matematica composta da un insieme di **nodi** (o vertici) V e un insieme di **archi** E , dove ogni arco collega una coppia di nodi. È lo strumento standard per rappresentare entità e le relazioni tra loro, quando quelle relazioni non hanno una struttura regolare a griglia (come i pixel di un'immagine) né sequenziale (come le parole di una frase): una rete di amicizie tra persone, le pagine web collegate da link, le molecole i cui atomi sono legati da legami chimici, e, in questo lavoro, una rete stradale, dove i nodi sono le intersezioni e gli archi le strade che le collegano direttamente.

Un arco può essere **diretto** (indica un verso, come " u segue v " sui social network) o **non diretto** (indica solo una relazione reciproca, come " u è amico di v "); può inoltre avere un **peso** associato, un numero che quantifica l'intensità o il costo della relazione (ad esempio la lunghezza di una strada). L'insieme dei nodi collegati direttamente a un nodo i da un arco si chiama **vicinato** di i , indicato $\mathcal{N}(i)$; il numero di archi incidenti su i è il suo **grado**. Una sequenza di nodi collegati uno al successivo da un arco è un **cammino**; la lunghezza del cammino più breve tra due nodi qualunque è la loro **distanza** sul grafo, misurata in numero di archi, non in senso fisico o geografico: due intersezioni collegate da una singola strada molto lun-

ga sono a distanza 1 tanto quanto due collegate da una strada brevissima. Questa nozione di distanza torna utile più avanti, quando si discuterà quanti passi di propagazione servano perché l'informazione di un nodo raggiunga un altro nodo della rete. Un grafo con I nodi si può rappresentare in forma matriciale con la **matrice di adiacenza** $A \in \{0, 1\}^{I \times I}$, dove $A_{iu} = 1$ se e solo se esiste un arco tra i e u (o il peso dell'arco, se il grafo è pesato, invece del semplice 1); A è simmetrica se e solo se il grafo non è diretto. È spesso utile aggiungere a ogni nodo un **self-loop**, un arco da un nodo verso se stesso ($A_{ii} = 1$), cosicché un nodo sia incluso nel proprio stesso vicinato: la notazione $\hat{A} = A + I$ (con I qui la matrice identità, da non confondere con il numero di nodi) indica questa aggiunta e ricorre nelle equazioni seguenti.

Un piccolo esempio concreto: tre intersezioni disposte in fila, $1 - 2 - 3$, dove 2 è collegata sia a 1 sia a 3, ma 1 e 3 non sono collegate direttamente. Il vicinato di 2 è $\mathcal{N}(2) = \{1, 3\}$, il suo grado è 2; i vicini di 1 e 3 sono invece $\mathcal{N}(1) = \{2\}$ e $\mathcal{N}(3) = \{2\}$, entrambi di grado 1. La matrice di adiacenza corrispondente, non diretta e senza pesi, è

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix},$$

simmetrica perché il grafo non è diretto: se 2 è vicino di 1, anche 1 è vicino di 2. Con l'aggiunta dei self-loop, $\hat{A} = A + I$ ha inoltre un 1 su ogni elemento della diagonale.

Nella rete stradale di questo lavoro, ogni nodo i del grafo porta con sé un vettore di caratteristiche (o *embedding*) che ne descrive lo stato corrente, ad esempio quanti veicoli attendono su ciascuna corsia: il problema che le GNN risolvono è come far sì che l'embedding di un nodo tenga conto anche di ciò che accade nei nodi a lui collegati, non solo del proprio stato isolato.

2.5.2 Message passing

Le *Graph Neural Network* (GNN) affrontano questo problema con un principio comune, il **message passing** (Wu et al., 2021; Bacciu et al., 2020). A ogni iterazione (o *layer*) l , ogni nodo i con rappresentazione corrente $h_i^{(l)}$ esegue due operazioni in successione: prima un passo di **aggregazione**, che combina i messaggi ricevuti da ciascun vicino $u \in \mathcal{N}(i)$ in un unico vettore di messaggio m_i ,

$$m_i = \text{AGGREGATE}(\{h_u^{(l)} : u \in \mathcal{N}(i)\}),$$

poi un passo di **aggiornamento**, che combina il messaggio aggregato con la rappresentazione precedente dello stesso nodo per produrre quella nuova,

$$h_i^{(l+1)} = \text{UPDATE}(h_i^{(l)}, m_i).$$

Tutte le GNN discusse in questa tesi, incluse le due formulazioni di seguito, sono istanze di questo stesso schema generale, che differiscono solo nella scelta concreta di AGGREGATE e UPDATE: nella GCN, AGGREGATE è una media pesata dalla sola topologia; nella GAT, un peso appreso in base al contenuto dei due nodi coinvolti. Impilando più layer, l'informazione può viaggiare oltre il solo vicinato immediato: dopo k layer, la rappresentazione di un nodo dipende (indirettamente) da tutti i nodi raggiungibili entro k archi di distanza, poiché il messaggio ricevuto da un vicino al layer l incorpora già, ricorsivamente, i messaggi che quel vicino aveva ricevuto dai propri stessi vicini al layer $l - 1$.

Graph Convolutional Network (GCN). Kipf and Welling (2017) propongono la formulazione più basilare di message passing tra quelle qui discusse: la rappresentazione di ciascun nodo si aggiorna mediando quelle dei vicini con un peso fisso, determinato unicamente dalla topologia locale (quanti vicini ha un nodo), non dal contenuto delle rappresentazioni stesse. Indicando con $\hat{A} = A + I$ la matrice di adiacenza con self-loop e con $\hat{D}$ la matrice diagonale dei gradi corrispondente ($\hat{D}_{ii} = \sum_u \hat{A}_{iu}$), la regola di propagazione di un singolo layer si scrive, in forma matriciale:

$$H^{(l+1)} = \sigma(\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} H^{(l)} W^{(l)}) \quad (2.5)$$

dove $H^{(l)} \in \mathbb{R}^{I \times d_l}$ raccoglie le rappresentazioni di tutti gli I nodi del grafo al layer l , $W^{(l)}$ è la matrice di pesi condivisa da ogni nodo e σ una non linearità puntuale. Per un singolo nodo i , la normalizzazione simmetrica $\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2}$ equivale a pesare il contributo di ogni vicino u con $1/\sqrt{\hat{d}_i \hat{d}_u}$:

$$h'_i = \sigma \left(\sum_{u \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\hat{d}_i \hat{d}_u}} W h_u \right) \quad (2.6)$$

un peso identico per ogni arco del vicinato di i , funzione solo del grado dei due nodi coinvolti e mai del loro contenuto. Tornando all'esempio a tre nodi 1 – 2 – 3 di §2.5, i gradi con self-loop sono $\hat{d}_1 = \hat{d}_3 = 2$ e $\hat{d}_2 = 3$: il nodo centrale 2 aggrega il proprio vicinato $\{1, 2, 3\}$ pesando ciascun contributo rispettivamente $1/\sqrt{\hat{d}_2 \hat{d}_1} = 1/\sqrt{6}$, $1/\sqrt{\hat{d}_2 \hat{d}_2} = 1/3$ e $1/\sqrt{\hat{d}_2 \hat{d}_3} = 1/\sqrt{6}$: due pesi identici verso i vicini 1 e 3, indipendentemente da quale dei due porti, ad esempio, il segnale di una coda di veicoli

più lunga. È, per costruzione, il termine di paragone più naturale rispetto a cui misurare cosa aggiunge un meccanismo di attenzione appreso come quello discusso di seguito: il Capitolo 3 confronta sperimentalmente proprio questa formulazione (MetaSTGCN) con l'attenzione appresa del modello di riferimento.

Graph Attention Network (GAT). Veličković et al. (2018) sostituiscono il peso fisso della GCN con un coefficiente di attenzione appreso, specifico per ciascuna coppia di nodi collegati: il contributo di un vicino non dipende più soltanto dalla topologia, ma anche dal suo contenuto. Dato un nodo i con vicinato $\mathcal{N}(i)$, embedding di input h_i , matrice di proiezione condivisa W e vettore di attenzione $\vec{a}$:

$$e_{iu} = \text{LeakyReLU}\left(\vec{a}^\top [Wh_i \parallel Wh_u]\right), \quad \alpha_{iu} = \frac{\exp(e_{iu})}{\sum_{u' \in \mathcal{N}(i)} \exp(e_{iu'})} \quad (2.7)$$

$$h'_i = \sigma\left(\sum_{u \in \mathcal{N}(i)} \alpha_{iu} Wh_u\right) \quad (2.8)$$

dove $\parallel$ denota la concatenazione: e_{iu} è un punteggio di compatibilità grezzo tra i e u , che il *softmax* sul vicinato trasforma in un coefficiente $\alpha_{iu} \in [0, 1]$ che somma a uno su tutti i vicini di i , esattamente come una distribuzione di probabilità. Con K teste di attenzione indipendenti (*multi-head attention*), ciascuna con i propri pesi W e $\vec{a}$, gli output si concatenano nei layer intermedi e si mediano nell'ultimo, secondo lo schema introdotto per l'attenzione da Vaswani et al. (2017): teste diverse possono così imparare a specializzarsi su aspetti diversi della relazione tra due nodi. A differenza della GCN, il coefficiente α_{iu} consente a un nodo di ponderare diversamente i propri vicini in base al loro contenuto, non solo alla loro posizione nel grafo: una proprietà rilevante per un'intersezione i cui vicini abbiano importanza marcatamente diversa, come nel caso di un'arteria principale contrapposta a una strada secondaria. La funzione di compatibilità additiva appena descritta, per quanto la più nota, non è l'unica possibile: il Capitolo 3 (Meta-GAT) ne impiega una variante strutturalmente diversa, con Query, Key e Value esplicite e distinte al posto della proiezione W condivisa, e un prodotto scalare al posto della concatenazione.

2.5.3 Invarianza e equivarianza per permutazione

Sia l'aggregazione della GCN sia quella della GAT condividono una proprietà che vale la pena rendere esplicita, perché è ciò che permette a queste reti di funzionare su grafi di dimensione diversa senza dover riaddestrare nulla. Entrambe calcolano il messaggio aggregato m_i di un nodo i come una somma (pesata, uniformemente

nella GCN o per attenzione nella GAT) sui vicini $u \in \mathcal{N}(i)$: una somma non dipende dall'ordine in cui i suoi addendi vengono elencati, quindi m_i non dipende dall'ordine in cui i vicini di i vengono numerati o elencati, ma solo da quali essi siano. Questa proprietà si chiama **invarianza per permutazione** dell'operatore di aggregazione, ed è ciò che distingue strutturalmente una GNN da una rete feedforward applicata a un vettore di caratteristiche di lunghezza fissa: una MLP a cui si desse in input, in un qualche ordine arbitrario, le caratteristiche di ciascun nodo del grafo concatenate assegnerebbe implicitamente un significato specifico a ciascuna posizione del vettore (il primo blocco di ingressi, il secondo, ...) e produrrebbe quindi un output diverso se semplicemente si rinumerassero i nodi, pur descrivendo esattamente la stessa rete stradale; Wei et al. (2019) osservano esattamente questo limite per gli schemi di coordinamento basati su una lista ordinata di vicini (§2.6). Applicando la stessa proprietà a ogni nodo del grafo simultaneamente, l'intero layer di message passing è **equivariante per permutazione**: rinumerare i nodi del grafo in ingresso produce in uscita esattamente le stesse rappresentazioni, solo rinumerate allo stesso modo, invece di un risultato arbitrariamente diverso (Bronstein et al., 2021).

Questa proprietà ha una conseguenza pratica diretta per la generalizzazione discussa nel Capitolo 4: le matrici di peso W di un layer GCN o GAT non fanno alcuna assunzione sul numero di nodi del grafo I o sul grado di un nodo specifico, poiché la stessa operazione di aggregazione, con gli stessi pesi, si applica indipendentemente al vicinato di ciascun nodo, quale che sia la sua cardinalità. Una rete addestrata su una griglia 4×4 (16 nodi) può quindi essere applicata, senza alcuna modifica ai pesi né riaddestramento, a una griglia 5×5 o 6×6 (25 o 36 nodi): l'unico requisito è che il vicinato di un incrocio in una griglia più grande assomigli, dal punto di vista della distribuzione delle caratteristiche osservate, a quello visto in addestramento. È esattamente questa proprietà, non un meccanismo aggiunto appositamente, a rendere possibile in linea di principio l'intera valutazione di generalizzazione a topologie di rete più grandi che il Capitolo 4 conduce su MetaSTGAT e sulle sue varianti.

2.5.4 Reti spatio-temporali su grafo

Una rete stradale non è soltanto un grafo: è un grafo il cui stato (occupazione delle corsie, fase attiva) evolve nel tempo, e le due dimensioni risultano intrecciate, poiché il valore osservato a un nodo in un dato istante è correlato sia con i valori passati allo stesso nodo sia con quelli ai nodi limitrofi. La famiglia di modelli che combina un layer di propagazione sul grafo (GCN o GAT, appena visti) con una componente ricorrente (LSTM, §2.4) sull'asse temporale prende il nome di *spatiotemporal graph neural network* (STGNN), un termine ormai consolidato nel-

la letteratura sul forecasting di serie temporali multivariate correlate (Cini et al., 2025). MetaSTGAT, descritto nel dettaglio nel Capitolo 3, costituisce un esempio di questa famiglia applicata al controllo semaforico anziché alla previsione: un layer di *graph attention* cattura la dipendenza spaziale tra intersezioni limitrofe, un layer ricorrente (LSTM) ne cattura l'evoluzione temporale, e i due vengono addestrati congiuntamente end-to-end.

2.5.5 Pesì generati da meta-learning

Sia la GCN sia la GAT, nella loro formulazione originale, condividono gli stessi pesi su tutti i nodi del grafo (esattamente il *parameter sharing* introdotto in §2.3.7): un'assunzione ragionevole quando i nodi sono sufficientemente omogenei, meno quando, come in una rete stradale reale, la topologia locale di ciascun nodo (numero di strade entranti, presenza di un'arteria, posizione periferica o centrale) può differire sensibilmente da nodo a nodo. Un *hypernetwork* affronta il problema generando i pesi di un layer non come parametri fissi appresi direttamente, bensì come output di una seconda rete, condizionata sulle caratteristiche locali del nodo: è di fatto una via di mezzo tra pesi completamente condivisi e pesi completamente indipendenti per ogni nodo, che riduce il costo del parameter sharing senza rinunciare del tutto ai suoi benefici. Il Capitolo 3 descrive nel dettaglio come MetaSTGAT applichi questo principio sia al layer di *graph attention* sia al layer ricorrente.

2.5.6 Propagazione a lungo raggio

Sia la GCN sia la GAT propagano informazione a un solo passo di distanza (un arco nel grafo) per layer: raggiungere un vicino a distanza k richiede impilarne altrettanti, con un costo di parametri crescente e un rischio noto di *over-smoothing*, per il quale le rappresentazioni dei nodi divengono troppo omogenee al crescere della profondità (ogni nodo finisce per "vedere" praticamente tutto il grafo, perdendo la capacità di distinguersi dai propri vicini). Meccanismi più recenti, tra cui **SONAR** (Trenta et al., 2025), affrontano il problema della propagazione a lungo raggio secondo un principio diverso, ispirato a sistemi dinamici oscillatori piuttosto che alla concatenazione sequenziale di layer: il dettaglio di questo meccanismo e la sua applicazione al controllo semaforico sono rimandati al Capitolo 3.

Un esempio numerico chiarisce perché il numero di passi L di propagazione (il parametro che, in un meccanismo come SONAR, gioca il ruolo che altrove è giocato dal numero di layer impilati) sia una scelta tutt'altro che neutra. In una griglia stradale come quella 4×4 usata per l'addestramento in questo lavoro, un'intersezione

interna ha 4 vicini diretti (Nord, Sud, Est, Ovest): a distanza 1 nel senso definito in §2.5 si trovano quindi fino a 4 nodi. A distanza 2 si aggiungono i vicini dei vicini: fino a 8 nodi ulteriori (i 4 vicini diagonali più altre 4 intersezioni allineate a due passi), per un totale di 12. In generale, su una griglia regolare il numero di nodi raggiungibili entro L passi cresce quadraticamente con L , non linearmente: passare da $L = 2$ a $L = 4$ non raddoppia il campo recettivo di un'intersezione, lo espande di un fattore prossimo a 4. Un campo recettivo più ampio non è però un vantaggio gratuito. Ogni passo aggiuntivo di propagazione richiede alla rete di retropropagare il gradiente attraverso uno stadio computazionale in più, e aumenta il rischio di *over-smoothing* discusso sopra: le rappresentazioni dei nodi coinvolti in un raggio via via più ampio tendono a somigliarsi, rendendo più difficile per il modello distinguere un'intersezione congestionata da una vicina che non lo è. Il Capitolo 4 riprende esattamente questo compromesso, confrontando empiricamente MetaSTSONAR con $L = 2$ e $L = 4$.

2.6 Traffic Signal Control come problema

2.6.1 Vocabolario di base

Il controllo semaforico (*Traffic Signal Control*, TSC) governa il transito dei veicoli in un'intersezione stradale. Il vocabolario di base, qui ripreso da [Wei et al. \(2020\)](#), è il seguente. Un' **approach** (approccio) è una via che porta all'intersezione, composta da una o più corsie in ingresso e, tipicamente, altrettante in uscita. Un **movimento** è il passaggio di un veicolo da una corsia in ingresso a una specifica corsia in uscita (ad esempio: prosecuzione dritto, svolta a destra o a sinistra), e a ogni movimento è associato un **segnale**, verde se il movimento è autorizzato, rosso altrimenti. Non tutti i movimenti possono ricevere il verde simultaneamente: due movimenti le cui traiettorie si intersecano fisicamente sono in **conflitto**, e la loro compatibilità reciproca si registra in una *matrice dei conflitti*. Un insieme di movimenti a due a due compatibili, che possono quindi ricevere il verde nello stesso istante, definisce una **fase**; il Capitolo 3 riporta le otto fasi usate in questo lavoro su un'intersezione a quattro bracci. Una **sequenza di fasi** stabilisce l'ordine in cui le fasi si succedono nel tempo, e un **piano semaforico** ne fissa anche la durata; quando la sequenza si ripete identica nel tempo si parla di piano **ciclico**, con **ciclo** la durata di una ripetizione completa.

2.6.2 Tre generazioni di controllo

Il TSC si articola in tre approcci, in ordine crescente di complessità e, storicamente, di introduzione (Shams et al., 2018).

Controllo a tempi fissi. Un piano semaforico ciclico, con fasi di durata predefinita calcolata su stime di traffico medie, si ripete senza mai osservare lo stato reale della rete. È un approccio robusto, in quanto non può fallire per un sensore guasto, ma per costruzione non reattivo: la stessa sequenza viene eseguita indipendentemente da congestioni anomale, incidenti o variazioni del traffico non modellate in fase di pianificazione. Metodi classici dell'ingegneria dei trasporti, come quello di Webster per una singola intersezione isolata, calcolano analiticamente la durata di ciclo e la ripartizione delle fasi che minimizza il tempo di attesa atteso sotto un'assunzione di flusso stazionario (Wei et al., 2020), un'ipotesi che nella pratica vale raramente per l'intera giornata. Il metodo di Webster, ad esempio, calcola la durata di ciclo desiderata come

$$C_0 = \frac{1.5L + 5}{1 - Y},$$

dove L è il tempo perso complessivo per ciclo (la somma dei brevi intervalli in cui nessun movimento riceve il verde, ad esempio durante il cambio di fase) e Y è la somma, su tutte le fasi, del rapporto tra flusso osservato e flusso di saturazione della fase più critica: un ciclo più lungo riduce l'incidenza percentuale del tempo perso ma allunga l'attesa media di chi deve aspettare il proprio turno, il compromesso centrale di ogni piano a tempi fissi. Il piano risultante resta valido solo finché il traffico reale resta vicino ai volumi Y usati per calcolarlo.

Controllo reattivo. Una regola locale osserva lo stato corrente dell'intersezione (tipicamente il numero di veicoli in coda per corsia) e sceglie l'azione che ottimizza un criterio istantaneo, senza affidarsi a un piano prestabilito. All'interno di questa categoria, Shams et al. (2018) distinguono ulteriormente il grado di adattività: il controllo *attuato* più semplice si limita a prolungare o abbreviare le fasi di un piano di base ancora sostanzialmente predefinito (estendere una fase finché un sensore rileva veicoli sulla corsia servita, ad esempio), mentre un controllo pienamente *adattivo* ricalcola la struttura stessa del piano, non solo la sua durata, in risposta al traffico osservato. Piattaforme come SCATS e SCOOT si collocano in questa seconda fascia, coordinando intere aree urbane aggiustando periodicamente i piani semaforici sulla base del traffico osservato, pur restando ancorate a una struttura di piano preesistente (Wei et al., 2020). MAX-PRESSURE (Varaiya, 2013) rappresenta invece una rottura netta con questa impostazione: modellando il movimento dei veicoli

come una rete di code *store-and-forward* (ogni corsia è una coda che riceve veicoli da monte e li inoltra a valle), definisce la **pressione** di un movimento $(r_i \rightarrow r_o)$, dalla corsia in ingresso r_i alla corsia in uscita r_o , come la differenza tra il numero di veicoli in coda sulle due corsie, $p(r_i \rightarrow r_o) = n(r_i) - n(r_o)$, dove $n(\cdot)$ conta i veicoli presenti sulla corsia: un valore positivo indica che la corsia a monte è più congestionata di quella a valle, quindi che servire quel movimento libererebbe più coda di quanta se ne creerebbe. La pressione di un'intera fase ϕ , che autorizza più movimenti compatibili contemporaneamente (§2.6), è la somma delle pressioni dei movimenti che quella fase abilita, $P(\phi) = \sum_{(r_i \rightarrow r_o) \in \phi} p(r_i \rightarrow r_o)$; l'algoritmo sceglie a ogni istante la fase $\operatorname{argmax}_{\phi} P(\phi)$, senza alcun piano né ciclo predefinito. Il risultato teorico centrale dell'articolo originale è che Max-Pressure è *massimamente stabilizzante*: se è possibile stabilizzare la rete, cioè non far divergere la lunghezza delle code nel tempo, Max-Pressure la stabilizza, usando solo la lunghezza delle code sulle corsie adiacenti all'intersezione e senza conoscere a priori il volume di domanda in ingresso. Max-Pressure è tuttavia miope per costruzione: non pianifica alcun compromesso su un orizzonte temporale non immediato, e la condizione di stabilità non tiene in considerazione le attese tra le diverse direzioni servite da un'intersezione, un aspetto di equità che questo lavoro esamina esplicitamente (Capitolo 4).

Controllo appreso. L'approccio basato sul reinforcement learning (§2.3) consente l'ottimizzazione diretta di metriche di rete globali (es. *travel time* o percentuale di completamento) sostituendo euristiche come la pressione istantanea, al netto del costo computazionale dell'addestramento simulato. Applicato a una rete di più intersezioni, il problema si formalizza come un Dec-POMDP (§2.3.7): ogni intersezione osserva solo il proprio stato locale, ma le sue azioni influenzano, con un ritardo proporzionale alla distanza spaziale, lo stato delle intersezioni adiacenti alterando il flusso di veicoli che le attraversano. **Questo lavoro si colloca interamente in questa terza categoria:** ogni intersezione è un agente RL model-free (§2.3.3) che decide quale fase attivare osservando solo il proprio stato locale.

Architetture di coordinamento. Esistono tre strategie distinte che affrontano la dipendenza tra intersezioni vicine. Un agente centralizzato che decide le azioni di tutta la rete con un'unica politica elimina il problema per costruzione, ma lo spazio delle azioni congiunte cresce esponenzialmente con il numero di intersezioni ed è impraticabile oltre poche decine di nodi. All'estremo opposto, un agente indipendente per intersezione, che non condivide alcuna informazione con i propri vicini: questo approccio scala senza difficoltà ma tratta ogni intersezione come se fosse isolata, ignorando che le sue azioni si propagano alla rete circostante. La via intermedia,

adottata anche da questo lavoro, fa comunicare agenti indipendenti condividendo informazione locale (architettura DTDE, §2.3.7). La difficoltà, discussa da [Wei et al. \(2019\)](#), risiede nel fatto che la trasmissione dello stato dei vicini in una sequenza predefinita (una lista ordinata di vicini) impone a tutte le intersezioni della rete gli stessi pesi interpretativi per ciascuna posizione della lista, senza tenere conto della reale importanza che ogni specifico nodo adiacente può rivestire per le diverse intersezioni del sistema. Una GNN (§2.5) evita il problema: pesando il contributo di ogni vicino in base al suo contenuto invece che alla sua posizione in un elenco, permette a un unico insieme di pesi condiviso di generalizzare tra intersezioni con topologie locali diverse, come quelle della rete adottata in questo lavoro (Capitolo 3).

Coordinamento oltre l’adiacenza fisica. Il grafo usato da MetaSTGAT e da questo lavoro coincide con l’adiacenza fisica della rete stradale: un’intersezione comunica solo con le intersezioni immediatamente collegate da una strada. Dalla letteratura si osserva che interazioni tra sole intersezioni adiacenti non siano sempre sufficienti a catturare tutte le dinamiche di traffico, e si propongono diverse soluzioni per ovviare a questo limite. Questo lavoro non affronta il tema direttamente, esplorandolo solo in parte nel Capitolo 4 attraverso un meccanismo di propagazione a lungo raggio (SONAR, §2.5) applicato comunque al solo grafo di adiacenza fisica. Il problema resta comunque aperto, e framework che combinano *graph attention*, una componente ricorrente sul tempo e reinforcement learning per il controllo semaforico continuano a essere oggetto di ricerca attiva.

Una tassonomia delle soluzioni recenti. La letteratura più recente sul reinforcement learning applicato al TSC classifica i metodi lungo assi in parte indipendenti da quelli discussi finora. Un primo asse distingue, all’interno del multi-agente, le architetture CTDE e DTDE appena introdotte. Un secondo asse raccoglie le tecniche di apprendimento integrate nel ciclo RL, ciascuna introdotta per rispondere a un limite specifico ([Xiao et al., 2025](#); [Wei et al., 2020](#)): reti neurali a grafo per ottenere una migliore coordinazione tra intersezioni, componenti ricorrenti per catturare la dipendenza tra passi successivi a livello temporale, transfer-learning per generalizzare rapidamente a scenari nuovi, meta-learning per adattarsi a condizioni operative diverse, reti generative avversarie per compensare dati di stato mancanti o incompleti. Nessuna di queste tecniche risolve da sola il problema: la letteratura le introduce quasi sempre in risposta a un limite preciso.

Dove si colloca MetaSTGAT. Il modello di questo lavoro è la combinazione di alcune delle tecniche appena citate. La sua è un’architettura multi-agente DT-

DE con scambio locale di informazione tramite GNN, parameter sharing attenuato dal meta-learning (§2.5), un componente ricorrente (LSTM) per catturare la dipendenza temporale oltre a quella spaziale (collocandosi nella famiglia delle reti spatio-temporali su grafo, STGNN), e un algoritmo di apprendimento model-free interamente basato sulla famiglia DQN (§2.3).

2.6.3 Simulazione

Valutare una politica di controllo semaforico su una rete reale non è praticabile durante lo sviluppo: cambiare i tempi di un semaforo cittadino migliaia di volte per tentativi, come richiederebbe l'addestramento di un agente RL, non è ovviamente proponibile. Serve quindi un simulatore che riproduca la dinamica del traffico con un dettaglio sufficiente, pur mantenendo un costo computazionale accettabile per l'addestramento di centinaia di episodi.

I simulatori di traffico si collocano su uno spettro di dettaglio a tre livelli. Un simulatore **macroscopico** rappresenta il traffico come un flusso continuo, descritto da grandezze aggregate come densità e velocità media di un tratto stradale, senza modellare i singoli veicoli: adatto a reti molto estese, ma troppo grossolano per catturare l'effetto di una singola decisione semaforica su una coda specifica. Un simulatore **microscopico** rappresenta invece ogni veicolo individualmente, con una propria posizione, velocità e traiettoria, aggiornate a ogni istante da un **modello di car-following** (che determina come un veicolo regola la propria velocità in base a quello che lo precede, tipicamente per evitare tamponamenti mantenendo una distanza di sicurezza) e da un modello di cambio di corsia: è il livello di dettaglio più fedele, ma anche il più costoso computazionalmente, poiché il numero di calcoli cresce con il numero di veicoli simulati, non con la sola dimensione della rete. Un simulatore **mesoscopico** si colloca a metà strada, modellando gruppi di veicoli o code aggregate per corsia invece del singolo veicolo, un compromesso diffuso proprio nel controllo semaforico basato su Max-Pressure (§2.6), dove il modello store-and-forward descritto sopra è esso stesso una forma di rappresentazione mesoscopica delle code.

CITYFLOW (Zhang et al., 2019) è il simulatore scelto in questo lavoro: è nominalmente microscopico (ogni veicolo ha una propria posizione e velocità), ma, rispetto ad alternative più diffuse come SUMO (Lopez et al., 2018), semplifica il modello di car-following e di cambio di corsia per privilegiare la velocità di simulazione rispetto al realismo del singolo veicolo, un compromesso adatto quando l'obiettivo primario è addestrare un agente su un numero elevato di episodi piuttosto che riprodurre con

precisione millimetrica la dinamica di un incrocio specifico. Il Capitolo 3 descrive nel dettaglio come questo lavoro configura l'ambiente CityFlow.

In sintesi, i quattro strumenti generali di questo capitolo compongono il modello di questo lavoro come segue: una rete neurale (§2.2) stima la funzione Q di un agente RL model-free (§2.3), uno per intersezione, in un sistema multi-agente coordinato tramite lo scambio locale di informazione proprio delle GNN (§2.5); una cella LSTM (§2.4) integra questa informazione spaziale con la sua evoluzione nel tempo; e il tutto è applicato al problema del controllo semaforico appena introdotto. Il Capitolo 3 riprende ciascuno di questi ingredienti uno a uno, componendoli nell'architettura completa.

Un filo conduttore attraversa più strumenti di questo capitolo, ed è utile renderlo esplicito: quello del **bias induttivo**, l'insieme di assunzioni che un modello incorpora prima ancora di osservare un solo dato, e che ne guidano la generalizzazione verso soluzioni plausibili invece che verso una qualunque delle infinite funzioni compatibili con l'esperienza osservata. Questo lavoro introduce almeno due bias induttivi espliciti, entrambi motivati dal problema applicativo piuttosto che scelti in astratto: l'equivarianza per permutazione della GNN (§2.5.3), che assume che il comportamento di un'intersezione non debba dipendere dall'ordine arbitrario in cui i suoi vicini vengono elencati, ed è la base della generalizzazione a griglie di dimensione diversa discussa nel Capitolo 4; e il termine di ricompensa dedicato all'attesa massima (Capitolo 3), che assume che nessuna direzione di traffico debba essere sacrificata indefinitamente, un'assunzione non implicita in una ricompensa che ottimizzasse il solo tempo di percorrenza aggregato. In entrambi i casi, il bias non è un limite imposto per semplificare il problema, ma una scelta di design che riflette direttamente uno degli obiettivi dichiarati di questo lavoro.

Capitolo 5

Conclusioni

Il Capitolo 1 ha dichiarato tre obiettivi per questo lavoro: efficienza, equità e generalizzazione. Questo capitolo li riprende uno a uno alla luce dei risultati del Capitolo 4, indica le direzioni lasciate aperte e discute infine i limiti dello studio.

Il confronto di base (§4.4.1) conferma che sia MaxPressure sia MetaSTGAT Pro battono nettamente Fixed-Time su ogni metrica, con l'unica eccezione del max wait in training, dove la rotazione fissa delle fasi di Fixed-Time impone per costruzione un limite all'attesa che le configurazioni di training, meno dense, non arrivano a violare: un limite che sparisce non appena il traffico si fa più intenso in test. Tra MaxPressure e Pro il compromesso al centro di questo lavoro emerge già a questo livello: MaxPressure resta la scelta più efficiente in media, Pro riduce il max wait del 34-41% pur restando entro pochi punti percentuali su travel time medio.

Lo studio sul peso α quantifica questo compromesso direttamente: max wait e $tt_{\max}$ peggiorano in modo pressoché monotono al diminuire di α , mentre il travel time medio non peggiora in modo altrettanto sistematico: $\alpha = 0.04$ lo riduce persino sotto quello di MaxPressure in test, il primo caso in questo lavoro in cui una variante RL supera MaxPressure su questa metrica in generalizzazione. Il caso più netto a sostegno della necessità del termine anti-starvation resta però $\alpha = 0$: rimuovere la sola penalità, lasciando intatti stato, maschera e vincolo di visibilità, è sufficiente da solo a produrre un blocco quasi totale di un veicolo per l'intera durata dell'episodio su quattro dei cinque seed della griglia 5×5 più densa mai vista in training. È una prova diretta, non solo correlazionale, che il termine di ricompensa (non lo stato, non la maschera) è il meccanismo causale dietro la garanzia di equità osservata. Per questo motivo, nonostante $\alpha = 0.04$ sia leggermente più efficiente in media, questo lavoro adotta $\alpha = 0.08$ come riferimento per gli studi successivi: coerentemente con la priorità dichiarata tra le metriche, è la configurazione più incisiva proprio sulle due metriche di caso peggiore che questo lavoro tiene in primo piano.

Lo studio di ablation isola un effetto complementare: dei cinque preset confrontati, solo i due che rimuovono interamente il segnale di attesa dallo stato o dalla ricompensa (**paper** ed **environment**) degradano marcatamente verso il comportamento di MaxPressure, mentre le tre varianti che disattivano singolarmente meccanismi dell’algoritmo di apprendimento (**temporal**, **single_dqn**, **vanilla_buffer**) restano vicine al riferimento o lo superano su alcune metriche. Su 35 istanze di test, **paper** ed **environment** sono gli unici, insieme a MaxPressure, a bloccare un veicolo per quasi l’intera durata dell’episodio; nessuna delle varianti che conserva stato e ricompensa lo fa mai. La garanzia di equità di questo lavoro dipende quindi dalla combinazione di segnale di attesa nello stato e termine di ricompensa dedicato, non da un singolo dettaglio implementativo dell’algoritmo di apprendimento sottostante.

Il confronto tra meccanismi spaziali aggiunge una terza componente necessaria: a parità di stato, ricompensa e maschera, MetaSTGCN (l’unico meccanismo a peso di aggregazione fissato dalla sola topologia, non appreso per coppia di nodi) resta sistematicamente indietro su ogni metrica, in un caso peggiore persino di MaxPressure nel proprio stesso regime di addestramento. Equità ed efficienza non sono quindi solo una funzione della ricompensa: richiedono anche un meccanismo di comunicazione capace di instradare il segnale del vicino in difficoltà fino a chi può agire su di esso.

Ogni modello di questo lavoro è addestrato una sola volta su una griglia 4×4 e valutato, senza riaddestramento, su topologie più grandi (5×5 , 6×6), su una spaziatura stradale diversa e su profili di traffico differenti. Il risultato principale è positivo ma non uniforme. Sulle configurazioni mai viste in training, MetaSTGAT Pro conserva il proprio vantaggio su MaxPressure e Fixed-Time su tutte le metriche principali, e le varianti che disattivano meccanismi dell’algoritmo di apprendimento (**single_dqn**, **vanilla_buffer**) mostrano un vantaggio sul riferimento concentrato quasi esclusivamente su queste topologie più grandi: alla scala di training, lo stesso vantaggio si riduce o si inverte a seconda della metrica, segno che quei meccanismi aiutano soprattutto la generalizzazione di scala, non le prestazioni in senso assoluto.

Un secondo asse di generalizzazione, la spaziatura stradale, rivela un meccanismo distinto: il preset **environment**, che tra le altre cose disattiva il vincolo sul campo visivo, degrada in modo sproporzionato proprio sulle due configurazioni con spaziatura di 200 metri, mai viste in training a nessuna spaziatura diversa da 100 metri. La causa più diretta è che quel vincolo limita per costruzione lo scarto tra la distribuzione di training e quella di test; senza di esso, il modello osserva conteggi di veicoli mai incontrati durante l’addestramento. Lo stesso pattern, attenuato, si osserva nel margine di vantaggio di **single_dqn** e **vanilla_buffer** sul max wait,

che si restringe quasi a scomparire sulle configurazioni a 200 metri pur restando ampio sulle griglie più grandi: generalizzare a una scala di rete diversa e generalizzare a una spaziatura fisica diversa non sono la stessa cosa, e i meccanismi che aiutano l’una non aiutano necessariamente l’altra.

Il caso di MetaSTSONAR chiude il quadro con un risultato meno atteso: a parità di ogni altro fattore, $L = 2$ ricorrenze di propagazione superano $L = 4$ su $tt_{\max}$ sia in training sia in test, quando il vantaggio teorico di un campo recettivo più ampio suggerirebbe l’opposto. Poiché lo svantaggio di $L = 4$ è già presente alla scala di training, dove non c’è alcun cambio di topologia da spiegare, la causa non può essere una generalizzazione peggiore: è più plausibile un limite legato all’ottimizzazione (il gradiente deve retropropagare attraverso il doppio dei passi dell’equazione di propagazione, a parità di procedura di training) o un effetto di *over-smoothing* già alla scala di training. Questo lavoro non arriva a isolare quale delle due spiegazioni, se non entrambe, sia corretta.

I risultati discussi finora aprono la strada a diverse direzioni di ricerca. Una prima direzione, di natura metodologica, è la ripetizione indipendente dell’addestramento per ciascuna configurazione: distinguere un effetto architetturale reale dalla varianza tra run indipendenti, specialmente per i risultati più sottili di questo lavoro (il vantaggio di `single_dqn` e `vanilla_buffer` sulla generalizzazione di scala, il comportamento di MetaSTSONAR a $L = 4$), richiederebbe ripetizioni non disponibili in questo studio. Una seconda direzione, sul confronto tra meccanismi spaziali, è un confronto a parametri pareggiati tra MetaSTGAT, MetaSTGCN e MetaSTSONAR, per isolare la capacità espressiva del meccanismo di aggregazione dal semplice numero di parametri disponibili. Una terza direzione, applicativa, è l’estensione della valutazione di generalizzazione a topologie stradali reali oltre alle griglie sintetiche di questo lavoro. Infine, i risultati sulla spaziatura stradale lasciano aperta una domanda diretta: se il divario di generalizzazione osservato dipenda specificamente dal vincolo sul campo visivo, come i risultati di `environment` suggeriscono, o più in generale da come lo stato rappresenta grandezze fisiche assolute invece che relative alla scala della rete; una domanda che uno studio dedicato, con configurazioni intermedie tra le spaziature qui testate, potrebbe risolvere in modo conclusivo dove questo lavoro si limita a un’osservazione.

Nonostante i risultati ottenuti, alcuni limiti vanno considerati nell’applicazione pratica di questo lavoro. Ogni modello, RL o ablation, deriva da un singolo run di addestramento: la batteria a più seed introdotta nel protocollo sperimentale ripete la sola valutazione, non l’addestramento, per cui buona parte dei risultati più sottili discussi in questo capitolo resta compatibile, oltre che con le spiegazioni proposte,

con la varianza tra run di addestramento indipendenti diversi dall'unico osservato. Il confronto tra meccanismi spaziali, come già notato, non pareggia il numero di parametri tra le tre famiglie confrontate: una parte del divario osservato, specialmente a sfavore di MetaSTGCN, potrebbe in principio riflettere anche questa differenza, non solo la capacità espressiva del meccanismo di aggregazione in sé. La valutazione di generalizzazione resta inoltre confinata a reti sintetiche a griglia regolare, generate parametricamente, mai a una rete stradale reale: un limite che questo lavoro condivide con la maggior parte della letteratura citata nell'introduzione, e che la sola estensione a griglie più grandi non risolve. Allo stesso modo, CITYFLOW privilegia la velocità di simulazione sul realismo microscopico del singolo veicolo (§2.6): risultati assoluti su un simulatore più dettagliato, o su dati reali, potrebbero differire pur lasciando invariate le conclusioni relative tra i controllori confrontati. Infine, il confronto tra modelli in questo lavoro non include un test di significatività statistica in senso classico: la consistenza osservata su più configurazioni di test e più seed di valutazione rende i risultati qualitativamente solidi, non un sostituto formale di un test di ipotesi.

Bibliografia

- Davide Bacciu, Federico Errica, Alessio Micheli, and Marco Podda. A gentle introduction to deep learning for graphs. *Neural Networks*, 129:203–221, 2020. doi: 10.1016/j.neunet.2020.06.006.
- Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- Andrea Cini, Ivan Marisca, Daniele Zambon, and Cesare Alippi. Graph deep learning for time series forecasting. *ACM Computing Surveys*, 57(12):321, 2025. doi: 10.1145/3742784.
- Zhiyuan Hu, Bowen Liu, Jianming Hu, Zihang Wang, Lihui Peng, and Yi Zhang. Mpnlight: A self-attention enhanced message passing graph neural network for multi-intersection traffic signal control. *Electronics*, 15(8):1655, 2026. doi: 10.3390/electronics15081655.
- Steven Kapturowski, Georg Ostrovski, John Quan, Remi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2019.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

- Yuxi Li. Deep reinforcement learning. *arXiv preprint arXiv:1810.06339*, 2018.
- Zachary C. Lipton, John Berkowitz, and Charles Elkan. A critical review of recurrent neural networks for sequence learning. *arXiv preprint arXiv:1506.00019*, 2015.
- Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. Microscopic traffic simulation using sumo. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018. doi: 10.1109/ITSC.2018.8569938.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Afshin Oroojlooy, Davood Hajinezhad, Mohammadreza Nazari, and Jorge Silva. Attentlight: Universal attention-based reinforcement learning model for traffic signal control. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.
- Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.
- Andalib Shams, A. M. Tahsin Emtenan, and Christopher M. Day. A taxonomy of adaptive traffic signal control. *IEEE Access*, 2018.
- Chao Sun, Yuhao Yang, Jiacheng Li, Weiyi Fang, and Peng Zhang. A multi-agent regional traffic signal control system integrating traffic flow prediction and graph attention networks. *Systems*, 14(1):47, 2026. doi: 10.3390/systems14010047.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5-rmsprop: Divide the Gradient by a Running Average of Its Recent Magnitude, 2012. COURSERA: Neural Networks for Machine Learning, lecture notes.
- Alessandro Trenta, Alessio Gravina, and Davide Bacciu. Sonar: Long-range graph propagation through information waves. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025.

- Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.
- Pravin Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195, 2013.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Jing Wang, Xiaopeng Wang, Yang Mo, Chao Huang, Yanan Li, Guanghe Li, and Hao Zhang. Attention-based spatio-temporal graph convolutional networks-enhanced deep reinforcement learning for adaptive traffic signal control in urban traffic networks. *Scientific Reports*, 2026. doi: 10.1038/s41598-026-61371-3.
- Min Wang, Libing Wu, Man Li, Dan Wu, Xiaochuan Shi, and Chao Ma. Meta-learning based spatial-temporal graph attention network for traffic signal control. *Knowledge-Based Systems*, 250:109166, 2022. doi: 10.1016/j.knosys.2022.109166.
- Hua Wei, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinshi Zang, Chacha Chen, Weinan Zhang, Yanmin Zhu, Kai Xu, and Zhenhui Li. Colight: Learning network-level cooperation for traffic signal control. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, 2019. doi: 10.1145/3357384.3357902.
- Hua Wei, Guanjie Zheng, Vikash Gayah, and Zhenhui Li. A survey on traffic signal control methods. *arXiv preprint arXiv:1904.08117*, 2020.
- Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021. doi: 10.1109/TNNLS.2020.2978386.
- Feng Xiao, Jiaming Lu, Lu Li, Wenwen Tu, and Chaojing Li. Advances in reinforcement learning for traffic signal control: a review of recent progress. *Intelligent Transportation Infrastructure*, 2025:liaf009, 2025. doi: 10.1093/iti/liaf009.

Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haili Jin, and Zhenhui Li. Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *The World Wide Web Conference (WWW)*, 2019. doi: 10.1145/3308558.3314139.

Huichu Zhang, Chang Liu, Weinan Zhang, Guanjie Zheng, and Yong Yu. Generalight: Improving environment generalization of traffic signal control via meta reinforcement learning. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1783–1792, 2020. doi: 10.1145/3340531.3411859.

Liang Zhang, Qiang Wu, Jun Shen, Linyuan Lu, Bo Du, and Jianqing Wu. Expression might be enough: representing pressure and demand for reinforcement learning based traffic signal control. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.